

**ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH**  
**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN**  
**KHOA KHOA HỌC MÁY TÍNH**



## **ĐỒ ÁN 1**

# **YOLO26: Kiến Trúc Phát Hiện Đối Tượng End-to-End Không Cần NMS**

**với Progressive Loss Balancing  
cho Hệ Thống Camera Tự Động  
Theo Dõi Diễn Giả**

**GVHD:** ThS. Trần Hoàng Lộc

**SVTH:** Sinh viên 1

Sinh viên 2

TP. Hồ Chí Minh, 2026

# Mục lục

|          |                                        |          |
|----------|----------------------------------------|----------|
| <b>1</b> | <b>Giới thiệu</b>                      | <b>1</b> |
| 1.1      | Bối cảnh và động lực                   | 1        |
| 1.2      | Mục tiêu đồ án                         | 1        |
| 1.3      | Phạm vi                                | 1        |
| 1.4      | Cấu trúc báo cáo                       | 1        |
| <b>2</b> | <b>Cơ sở lý thuyết</b>                 | <b>2</b> |
| 2.1      | Tổng quan Object Detection             | 2        |
| 2.1.1    | Phân loại kiến trúc                    | 2        |
| 2.2      | Intersection over Union (IoU)          | 2        |
| 2.2.1    | Generalized IoU (GIoU)                 | 3        |
| 2.2.2    | Distance IoU (DIoU)                    | 3        |
| 2.2.3    | Complete IoU (CIoU)                    | 3        |
| 2.3      | Precision, Recall và mAP               | 3        |
| 2.4      | Non-Maximum Suppression (NMS)          | 4        |
| 2.5      | Tích chập (Convolution)                | 4        |
| 2.6      | Batch Normalization                    | 4        |
| 2.7      | Hàm kích hoạt SiLU                     | 5        |
| 2.8      | Binary Cross-Entropy (BCE) Loss        | 5        |
| 2.9      | Feature Pyramid Network (FPN) và PANet | 5        |
| 2.10     | Residual Connection                    | 5        |
| 2.11     | Self-Attention                         | 6        |
| 2.12     | Anchor Points vs Anchor Boxes          | 6        |
| 2.13     | Dòng YOLO — Lịch sử phát triển         | 6        |
| 2.14     | Edge Computing và Raspberry Pi 5       | 7        |
| 2.15     | NCNN Framework                         | 7        |
| <b>3</b> | <b>Kiến trúc tổng thể YOLO26</b>       | <b>8</b> |
| 3.1      | Tổng quan kiến trúc                    | 8        |
| 3.2      | Hệ thống Scale                         | 8        |
| 3.3      | Backbone                               | 9        |
| 3.4      | Neck — FPN + PAN                       | 10       |
| 3.4.1    | Top-Down Pathway (FPN)                 | 10       |
| 3.4.2    | Bottom-Up Pathway (PAN)                | 10       |
| 3.4.3    | Upsample — Nearest Neighbor            | 11       |
| 3.4.4    | Concat                                 | 11       |
| 3.5      | Detection Head — Dual Head             | 11       |
| 3.5.1    | Box Head (cv2)                         | 11       |

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| 3.5.2    | Classification Head (cv3)                     | 11        |
| 3.5.3    | One-to-Many (O2M) Head                        | 11        |
| 3.5.4    | One-to-One (O2O) Head                         | 11        |
| 3.5.5    | Giải mã Bounding Box                          | 12        |
| 3.5.6    | Postprocessing (Inference)                    | 12        |
| 3.5.7    | Strip O2M khi export                          | 12        |
| 3.6      | Multi-Scale Detection                         | 12        |
| <b>4</b> | <b>Các khối Module chi tiết</b>               | <b>13</b> |
| 4.1      | Conv — Convolution Block cơ bản               | 13        |
| 4.2      | DWConv — Depthwise Convolution                | 13        |
| 4.3      | Bottleneck — Khối Residual cơ bản             | 14        |
| 4.4      | C2f — Cross Stage Partial with 2 Convolutions | 14        |
| 4.5      | C3 và C3k                                     | 15        |
| 4.6      | C3k2 — Khối quan trọng nhất trong YOLO26      | 15        |
| 4.7      | SPPF — Spatial Pyramid Pooling Fast           | 17        |
| 4.8      | Attention — Multi-Head Self-Attention         | 18        |
| 4.9      | PSABlock — Positional Self-Attention Block    | 19        |
| 4.10     | C2PSA — Channel-to-Positional Self-Attention  | 20        |
| 4.11     | DFL — Distribution Focal Loss Module          | 21        |
| 4.12     | Tổng quan đồ thị module                       | 22        |
| <b>5</b> | <b>Hàm mất mát</b>                            | <b>23</b> |
| 5.1      | Tổng quan Loss trong YOLO26                   | 23        |
| 5.2      | Box Loss — CIoU Loss                          | 23        |
| 5.3      | Classification Loss — BCE Loss                | 23        |
| 5.4      | DFL/LTRB Loss                                 | 23        |
| 5.4.1    | Khi $reg\_max > 1$ : DFL Loss                 | 23        |
| 5.4.2    | Khi $reg\_max = 1$ : L1 Loss (YOLO26 default) | 24        |
| 5.5      | TaskAlignedAssigner (TAL)                     | 24        |
| 5.5.1    | Alignment Metric                              | 24        |
| 5.5.2    | Quy trình gán nhãn                            | 24        |
| 5.5.3    | STAL — Small-Target-Aware Label Assignment    | 25        |
| 5.6      | E2ELoss — End-to-End Loss                     | 25        |
| 5.7      | ProgLoss — Progressive Loss Balancing         | 25        |
| 5.8      | Công thức ghép đầy đủ                         | 26        |
| <b>6</b> | <b>Chiến lược huấn luyện</b>                  | <b>27</b> |
| 6.1      | Training Loop                                 | 27        |
| 6.2      | Optimizer — Phân nhóm tham số                 | 27        |
| 6.3      | Learning Rate Schedule                        | 28        |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| 6.4      | Warmup . . . . .                                | 28        |
| 6.5      | Exponential Moving Average (EMA) . . . . .      | 28        |
| 6.6      | Automatic Mixed Precision (AMP) . . . . .       | 28        |
| 6.7      | Gradient Clipping . . . . .                     | 29        |
| 6.8      | Gradient Accumulation . . . . .                 | 29        |
| 6.9      | Data Augmentation . . . . .                     | 29        |
| 6.10     | EarlyStopping . . . . .                         | 29        |
| 6.11     | Letterbox Resize . . . . .                      | 29        |
| <b>7</b> | <b>Muon Optimizer và MuSGD</b>                  | <b>31</b> |
| 7.1      | Vấn đề của SGD/AdamW truyền thống . . . . .     | 31        |
| 7.2      | Giải pháp: Trục giao hóa Gradient . . . . .     | 31        |
| 7.3      | Singular Value Decomposition (SVD) . . . . .    | 31        |
| 7.4      | Newton–Schulz Iteration . . . . .               | 32        |
| 7.4.1    | Bước 0: Chuẩn hóa Frobenius . . . . .           | 32        |
| 7.4.2    | Công thức lặp (Standard — Quadratic) . . . . .  | 32        |
| 7.4.3    | Dạng Cubic (dùng trong Muon/YOLO26) . . . . .   | 32        |
| 7.4.4    | Ý nghĩa đa thức . . . . .                       | 33        |
| 7.5      | Ví dụ tính toán chi tiết . . . . .              | 33        |
| 7.6      | Tốc độ hội tụ . . . . .                         | 34        |
| 7.7      | MuSGD — Hybrid Optimizer trong YOLO26 . . . . . | 34        |
| 7.8      | Tóm tắt quy trình Muon . . . . .                | 35        |
| <b>8</b> | <b>Dataset và Triển khai</b>                    | <b>38</b> |
| 8.1      | Tập dữ liệu COCO . . . . .                      | 38        |
| 8.1.1    | Format nhãn YOLO . . . . .                      | 38        |
| 8.1.2    | Tiền xử lý dữ liệu . . . . .                    | 38        |
| 8.2      | Cấu hình huấn luyện . . . . .                   | 39        |
| 8.3      | Triển khai trên Raspberry Pi 5 . . . . .        | 39        |
| 8.3.1    | Pipeline Export . . . . .                       | 39        |
| 8.3.2    | NCNN Runtime . . . . .                          | 39        |
| 8.3.3    | So sánh với khóa trước . . . . .                | 40        |
| 8.4      | Inference Pipeline . . . . .                    | 40        |
| <b>9</b> | <b>Kết quả thực nghiệm</b>                      | <b>41</b> |
| 9.1      | Phân tích quá trình huấn luyện . . . . .        | 41        |
| 9.1.1    | Diễn biến Loss và mAP . . . . .                 | 41        |
| 9.1.2    | Dấu hiệu Overfitting . . . . .                  | 41        |
| 9.1.3    | Sự cố Epoch 52 . . . . .                        | 42        |
| 9.2      | Kiểm chứng kiến trúc . . . . .                  | 42        |
| 9.3      | Các vấn đề phát hiện . . . . .                  | 42        |

|           |                                             |           |
|-----------|---------------------------------------------|-----------|
| 9.3.1     | reg_max=1 vô hiệu hóa DFL . . . . .         | 42        |
| 9.3.2     | Data Augmentation yếu . . . . .             | 42        |
| 9.3.3     | E2ELoss decay quá mạnh . . . . .            | 42        |
| 9.4       | Benchmark FPS trên Raspberry Pi 5 . . . . . | 43        |
| <b>10</b> | <b>Kết luận và hướng phát triển</b>         | <b>44</b> |
| 10.1      | Tóm tắt đóng góp . . . . .                  | 44        |
| 10.2      | Hạn chế . . . . .                           | 44        |
| 10.3      | Hướng phát triển . . . . .                  | 44        |
| <b>A</b>  | <b>Phụ lục: Code nguồn chính</b>            | <b>45</b> |
| A.1       | Kiến trúc YOLO26n . . . . .                 | 45        |
| A.2       | make_anchors . . . . .                      | 45        |
| A.3       | dist2bbox và bbox2dist . . . . .            | 46        |
| A.4       | Newton–Schulz Iteration . . . . .           | 46        |
| A.5       | E2ELoss decay . . . . .                     | 47        |
|           | <b>Tài liệu tham khảo</b>                   | <b>48</b> |

# Danh sách hình vẽ

|      |                                                                                                             |    |
|------|-------------------------------------------------------------------------------------------------------------|----|
| 3.1  | Kiến trúc tổng thể YOLO26: Backbone $\rightarrow$ Neck (FPN+PAN) $\rightarrow$ Dual Detection Head. . . . . | 8  |
| 4.1  | Sơ đồ khối Bottleneck với skip connection. . . . .                                                          | 14 |
| 4.2  | Sơ đồ khối C3k — C3 với kernel 3x3 sâu hơn. . . . .                                                         | 15 |
| 4.3  | Sơ đồ khối C3k2 — forward theo C2f, block bên trong tùy cấu hình. . . . .                                   | 16 |
| 4.4  | Sơ đồ khối SPPF — MaxPool liên tiếp tạo multi-scale receptive field. . . . .                                | 17 |
| 4.5  | Mình họa MaxPool 5x5 qua 3 lần liên tiếp — tạo receptive field 5x5, 9x9, 13x13. . . . .                     | 17 |
| 4.6  | Sơ đồ khối Attention với QKV projection và Position Encoding. . . . .                                       | 18 |
| 4.7  | Chi tiết flow và shape biến đổi trong Attention module. . . . .                                             | 19 |
| 4.8  | Sơ đồ PSABlock: Attention + FFN với residual connections. . . . .                                           | 19 |
| 4.9  | Sơ đồ C2PSA: CSP splitting + PSABlock. . . . .                                                              | 20 |
| 4.10 | Biến đổi feature map qua SPPF và C2PSA. . . . .                                                             | 21 |
| 4.11 | Tổng quan các khối đặc biệt: C3k2, SPPF, C2PSA trong kiến trúc YOLO26. . . . .                              | 22 |
| 5.1  | ProgLoss curves. Trái: O2M/O2O weights. Phải: Box weight giảm, Cls weight tăng. . . . .                     | 26 |
| 7.1  | So sánh SVD exact vs Newton–Schulz: hội tụ singular values về 1. . . . .                                    | 34 |
| 7.2  | So sánh hiệu ứng SGD vs Muon trên cập nhật ma trận trọng số. . . . .                                        | 35 |
| 7.3  | SVD step-by-step: Xoay ( $V^T$ ) $\rightarrow$ Co giãn ( $\Sigma$ ) $\rightarrow$ Xoay ( $U$ ). . . . .     | 36 |
| 7.4  | Tại sao chuẩn hóa Frobenius: đảm bảo singular values $\leq 1$ cho hội tụ. . . . .                           | 37 |
| 9.1  | Biểu đồ loss và mAP qua 100 epoch huấn luyện YOLO26n. . . . .                                               | 41 |
| 9.2  | Kết quả dự đoán YOLO26n trên ảnh mẫu. . . . .                                                               | 43 |

# Danh sách bảng

|     |                                                                           |    |
|-----|---------------------------------------------------------------------------|----|
| 2.1 | Lịch sử phát triển dòng YOLO. . . . .                                     | 6  |
| 3.1 | Cấu hình Scale của YOLO26. . . . .                                        | 9  |
| 3.2 | Kiến trúc Backbone YOLO26n (scale nano, ảnh $320 \times 320$ ). . . . .   | 9  |
| 3.3 | So sánh model trước và sau khi strip O2M. . . . .                         | 12 |
| 3.4 | Số anchor points tại mỗi scale detection (ảnh $320 \times 320$ ). . . . . | 12 |
| 4.1 | Sử dụng C3k2 trong YOLO26. . . . .                                        | 16 |
| 5.1 | Trọng số các thành phần loss (thay đổi theo ProgLoss). . . . .            | 23 |
| 5.2 | Lịch trình trọng số O2M/O2O qua epoch. . . . .                            | 25 |
| 6.1 | Phân nhóm tham số optimizer. . . . .                                      | 27 |
| 6.2 | Data augmentation trong YOLO26. . . . .                                   | 29 |
| 7.1 | Hội tụ singular values qua các bước Newton–Schulz. . . . .                | 33 |
| 7.2 | Phân nhóm tham số trong MuSGD. . . . .                                    | 34 |
| 8.1 | Thông số tập dữ liệu. . . . .                                             | 38 |
| 8.2 | Hyperparameters huấn luyện YOLO26n. . . . .                               | 39 |
| 8.3 | So sánh hai phương pháp triển khai trên RPi. . . . .                      | 40 |
| 9.1 | Timeline phân tích overfitting qua các giai đoạn huấn luyện. . . . .      | 41 |
| 9.2 | Kết quả kiểm chứng kiến trúc YOLO26. . . . .                              | 42 |
| 9.3 | Benchmark FPS trên RPi 5, NCNN runtime, CPU only. *Cần re-verify. . . . . | 43 |

# CHƯƠNG 1

## Giới thiệu

### 1.1 Bối cảnh và động lực

Bài toán phát hiện đối tượng (Object Detection) yêu cầu mô hình đồng thời giải quyết hai tác vụ: **phân loại** — xác định đối tượng thuộc lớp nào, và **định vị** — xác định vị trí đối tượng qua bounding box.

Dòng YOLO [1] tạo bước đột phá khi đưa bài toán về dạng *single-shot*. YOLO26 [2] là phiên bản mới nhất, loại bỏ hoàn toàn NMS nhờ Dual Detection Head (O2M/O2O), kết hợp Progressive Loss Balancing và MuSGD optimizer.

### 1.2 Mục tiêu đề án

- 1) Nghiên cứu chi tiết kiến trúc YOLO26 từ Backbone đến Detection Head.
- 2) Xây dựng pipeline huấn luyện từ đầu bằng PyTorch thuần.
- 3) Huấn luyện và đánh giá mô hình trên tập COCO.
- 4) Triển khai trên Raspberry Pi 5 qua NCNN [3] đạt real-time detection.

### 1.3 Phạm vi

Tập trung vào detection (phát hiện người), mô hình YOLO26n với  $reg\_max=1$ , triển khai trên Raspberry Pi 5 với CPU thuần. Chưa tích hợp ByteTrack [4] cho tracking.

### 1.4 Cấu trúc báo cáo

Báo cáo gồm 10 chương: Giới thiệu, Cơ sở lý thuyết, Kiến trúc YOLO26, Các module chi tiết, Hàm loss, Huấn luyện, Muon Optimizer, Dataset và Triển khai, Kết quả, Kết luận.

## CHƯƠNG 2

# Cơ sở lý thuyết

### 2.1 Tổng quan Object Detection

Object Detection là bài toán kết hợp *phân loại* (classification) và *định vị* (localization) — xác định vị trí đối tượng trong ảnh thông qua bounding box.

#### 2.1.1 Phân loại kiến trúc

- **Two-stage detectors** (Faster R-CNN [5]): Đề xuất vùng ứng viên rồi phân loại. Độ chính xác cao nhưng chậm.
- **One-stage detectors** (YOLO, SSD): Dự đoán box và lớp trong một forward pass. Nhanh, phù hợp real-time.

Phân loại theo anchor:

- **Anchor-based**: Sử dụng anchor boxes với kích thước định trước.
- **Anchor-free**: Trực tiếp dự đoán tọa độ, không cần anchor. YOLO26 sử dụng anchor-free.

### 2.2 Intersection over Union (IoU)

#### Khái niệm: IoU

IoU đo mức độ trùng khớp giữa hộp dự đoán  $B_p$  và hộp thực tế  $B_{gt}$ :

$$\text{IoU} = \frac{|B_p \cap B_{gt}|}{|B_p \cup B_{gt}|} = \frac{\text{Diện tích giao}}{\text{Diện tích hợp}} \quad (2.1)$$

$\text{IoU} \in [0, 1]$ : giá trị 0 = không giao nhau, giá trị 1 = trùng khớp hoàn toàn.

**Tính toán cụ thể:** Cho hai box ở dạng  $(x_1, y_1, x_2, y_2)$ :

$$x_{\text{inter}}^{\min} = \max(x_1^{(p)}, x_1^{(gt)}), \quad y_{\text{inter}}^{\min} = \max(y_1^{(p)}, y_1^{(gt)}) \quad (2.2)$$

$$x_{\text{inter}}^{\max} = \min(x_2^{(p)}, x_2^{(gt)}), \quad y_{\text{inter}}^{\max} = \min(y_2^{(p)}, y_2^{(gt)}) \quad (2.3)$$

$$\text{Inter} = \max(x_{\text{inter}}^{\max} - x_{\text{inter}}^{\min}, 0) \times \max(y_{\text{inter}}^{\max} - y_{\text{inter}}^{\min}, 0) \quad (2.4)$$

$$\text{Union} = \text{Area}(B_p) + \text{Area}(B_{gt}) - \text{Inter} \quad (2.5)$$

### 2.2.1 Generalized IoU (GIoU)

GIoU [6] giải quyết vấn đề  $\text{IoU} = 0$  khi hai box không giao nhau:

$$\text{GIoU} = \text{IoU} - \frac{|C \setminus (B_p \cup B_{gt})|}{|C|} \quad (2.6)$$

$C$  là hình chữ nhật nhỏ nhất chứa cả  $B_p$  và  $B_{gt}$ .  $\text{GIoU} \in [-1, 1]$ .

### 2.2.2 Distance IoU (DIoU)

DIoU [7] thêm penalty dựa trên khoảng cách tâm:

$$\text{DIoU} = \text{IoU} - \frac{\rho^2(\mathbf{b}_p, \mathbf{b}_{gt})}{c^2} \quad (2.7)$$

$\rho(\mathbf{b}_p, \mathbf{b}_{gt})$  là khoảng cách Euclidean giữa tâm hai box,  $c$  là đường chéo của box bao  $C$ .

### 2.2.3 Complete IoU (CIoU)

**Khái niệm: CIoU — YOLO26 sử dụng CIoU làm loss chính**

CIoU [7] mở rộng IoU với penalty khoảng cách tâm và tỷ lệ khung hình:

$$\text{CIoU} = \text{IoU} - \frac{\rho^2(\mathbf{b}_p, \mathbf{b}_{gt})}{c^2} - \alpha v \quad (2.8)$$

trong đó:

$$v = \frac{4}{\pi^2} \left( \arctan \frac{w_{gt}}{h_{gt}} - \arctan \frac{w_p}{h_p} \right)^2 \quad (2.9)$$

$$\alpha = \frac{v}{(1 - \text{IoU}) + v} \quad (2.10)$$

$v$  đo sự khác biệt tỷ lệ khung hình,  $\alpha$  là trọng số cân bằng.

## 2.3 Precision, Recall và mAP

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (2.11)$$

$TP$  = True Positive (dự đoán đúng),  $FP$  = False Positive (dự đoán sai),  $FN$  = False Negative (bỏ sót). Một dự đoán được coi là  $TP$  khi  $\text{IoU} \geq$  ngưỡng (thường 0.50).

**Average Precision (AP):** Diện tích dưới đường cong Precision-Recall, nội suy 101 điểm.

$$\text{mAP} = \frac{1}{N_c} \sum_{i=1}^{N_c} \text{AP}_i \quad (2.12)$$

- **mAP@50**: AP tại ngưỡng IoU  $\geq 0.50$ .
- **mAP@50-95**: Trung bình AP tại 10 ngưỡng IoU từ 0.50 đến 0.95 (bước 0.05).

**Fitness metric** của YOLO26:  $\text{fitness} = 0.1 \times \text{mAP50} + 0.9 \times \text{mAP50-95}$ .

## 2.4 Non-Maximum Suppression (NMS)

### Khái niệm: NMS

NMS là bước hậu xử lý loại bỏ các box trùng lặp:

1. Sắp xếp detections theo confidence giảm dần
2. Chọn detection có score cao nhất, thêm vào kết quả
3. Loại bỏ tất cả detections có IoU  $>$  ngưỡng (thường 0.45)
4. Lặp lại cho đến hết

**Hạn chế**: Tốn thời gian  $O(n^2)$ , cần chọn ngưỡng thủ công, có thể loại bỏ nhầm detections hợp lệ khi đối tượng chồng lấp. **YOLO26 loại bỏ hoàn toàn NMS** nhờ cơ chế O2O head.

## 2.5 Tích chập (Convolution)

Phép tích chập 2D là phép toán cơ bản trong mạng neural xử lý ảnh:

$$y(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} x(i+m, j+n) \cdot w(m, n) + b \quad (2.13)$$

**Autopadding**: Tự động tính padding  $p = \lfloor k/2 \rfloor$  để giữ kích thước spatial khi stride = 1.

**Depthwise Convolution (DWConv)**: Conv với groups = gcd( $c_1, c_2$ ), xử lý từng channel độc lập. Giảm tham số:

$$\text{Params}_{\text{DW}} = c_1 \times k^2 \quad \text{vs} \quad \text{Params}_{\text{Std}} = c_1 \times c_2 \times k^2 \quad (2.14)$$

## 2.6 Batch Normalization

BN [8] chuẩn hóa đầu ra theo batch, ổn định quá trình huấn luyện:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \cdot \gamma + \beta \quad (2.15)$$

$\mu_B, \sigma_B^2$  là trung bình và phương sai theo batch.  $\gamma, \beta$  là tham số học được.

## 2.7 Hàm kích hoạt SiLU

SiLU [9] (Sigmoid Linear Unit) hay Swish:

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \quad (2.16)$$

So với ReLU, SiLU cho gradient khác không ở vùng âm, giúp mô hình học tốt hơn. SiLU được sử dụng xuyên suốt trong YOLO26.

## 2.8 Binary Cross-Entropy (BCE) Loss

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (2.17)$$

$y_i \in \{0, 1\}$  là nhãn thực,  $\hat{y}_i = \sigma(z_i)$  là xác suất qua sigmoid. YOLO26 dùng BCEWithLogitsLoss kết hợp sigmoid+BCE trong một bước.

## 2.9 Feature Pyramid Network (FPN) và PANet

**FPN** [10]: Tạo đặc trưng đa tỷ lệ bằng đường truyền top-down (semantic mạnh  $\rightarrow$  spatial chi tiết). Các tầng sâu (high-level) có semantic information nhưng spatial resolution thấp. FPN kết hợp cả hai.

**PANet** [11]: Bổ sung đường truyền bottom-up, giúp thông tin chi tiết từ tầng nông truyền xuống tầng sâu.

YOLO26 sử dụng Neck kết hợp cả FPN (top-down) và PAN (bottom-up) để tổng hợp đặc trưng đa tỷ lệ. Chi tiết kiến trúc được trình bày trong Chương 3.

## 2.10 Residual Connection

Residual connection [12] cho phép gradient lan truyền trực tiếp qua shortcut:

$$y = F(x) + x \quad (2.18)$$

Giải quyết vấn đề vanishing gradient khi mạng sâu. Được sử dụng trong Bottleneck, SPPF, PSABlock của YOLO26.

## 2.11 Self-Attention

Multi-Head Self-Attention [13] cho phép mỗi vị trí trên feature map “nhìn” toàn bộ các vị trí khác:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.19)$$

$Q, K, V$  là Query, Key, Value.  $d_k$  là chiều Key. Softmax chuẩn hóa attention weights. YOLO26 dùng self-attention ở tầng P5 (C2PSA, PSABlock) nơi feature map nhỏ ( $10 \times 10$ ).

## 2.12 Anchor Points vs Anchor Boxes

### Anchor Point — Kiến trúc Anchor-free

YOLO26 là kiến trúc **anchor-free**. Mỗi ô trên feature map có **một điểm neo (anchor point)** tại tâm, không có kích thước sẵn. Model dự đoán 4 khoảng cách ( $l, t, r, b$ ) từ anchor point đến 4 cạnh bounding box:

$$\text{anchor}_{i,j} = (j + 0.5, i + 0.5), \quad i \in [0, H), j \in [0, W) \quad (2.20)$$

Giải mã bounding box:

$$x_1 = a_x - l, \quad y_1 = a_y - t, \quad x_2 = a_x + r, \quad y_2 = a_y + b \quad (2.21)$$

## 2.13 Dòng YOLO — Lịch sử phát triển

| Phiên bản     | Năm         | Đóng góp chính                           |
|---------------|-------------|------------------------------------------|
| YOLOv1        | 2016        | Single-shot detection, Darknet backbone  |
| YOLOv2–v3     | 2017–18     | Multi-scale prediction, Darknet-53       |
| YOLOv4–v5     | 2020        | CSP backbone, Mosaic augmentation, PANet |
| YOLOX         | 2021        | Decoupled Head, Anchor-free              |
| YOLOv8        | 2023        | DFL (Distribution Focal Loss), C2f block |
| YOLO11        | 2024        | C3k2, C2PSA, multi-task                  |
| <b>YOLO26</b> | <b>2025</b> | NMS-free (O2M/O2O), ProgLoss, MuSGD      |

**Bảng 2.1:** Lịch sử phát triển dòng YOLO.

## 2.14 Edge Computing và Raspberry Pi 5

**Edge Computing** xử lý dữ liệu tại thiết bị đầu cuối, giảm độ trễ và tăng bảo mật. Raspberry Pi 5 [14] sử dụng CPU ARM Cortex-A76 @ 2.4GHz, 4/8GB RAM.

## 2.15 NCNN Framework

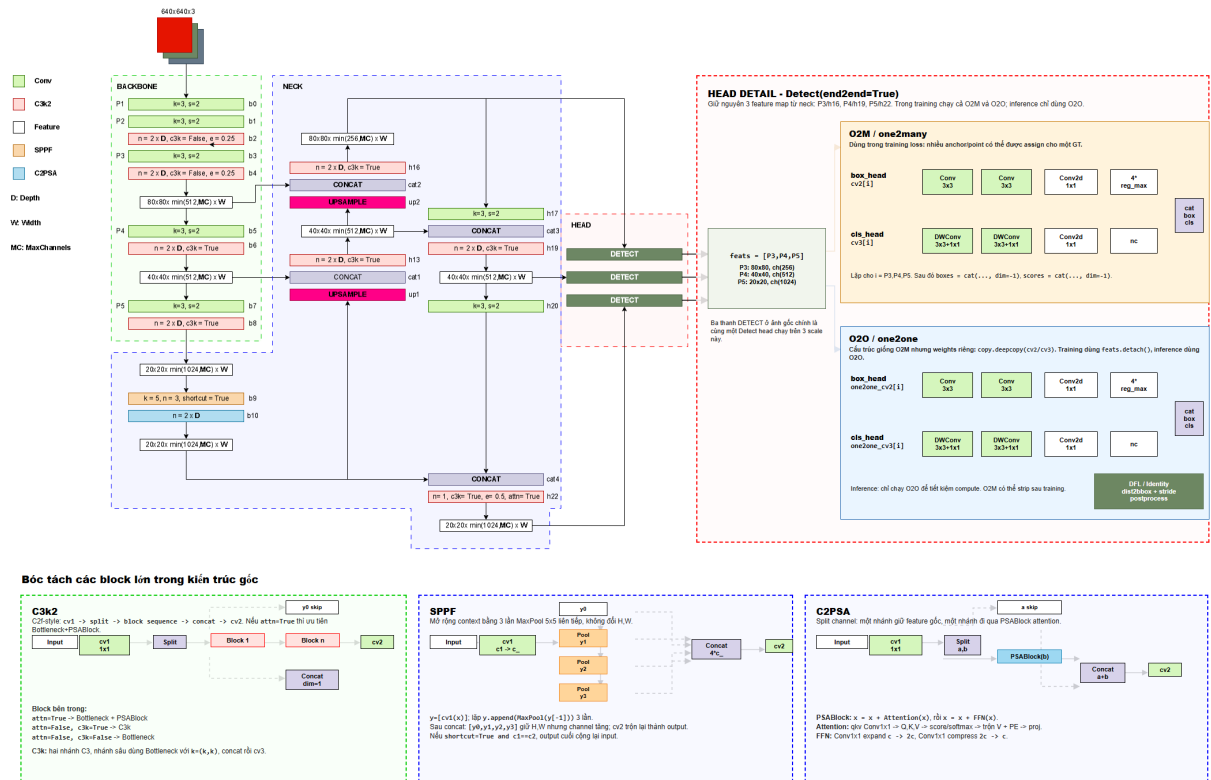
NCNN [3] là framework suy luận viết bằng C++, tối ưu NEON/SIMD cho ARM. So với TFLite [15], NCNN nhanh hơn trên CPU ARM nhờ tối ưu trực tiếp.

# CHƯƠNG 3

## Kiến trúc tổng thể YOLO26

### 3.1 Tổng quan kiến trúc

YOLO26 gồm 3 phần chính: **Backbone** (trích xuất đặc trưng), **Neck** (tổng hợp đa tỷ lệ), và **Detection Head** (dự đoán box + class).



Hình 3.1: Kiến trúc tổng thể YOLO26: Backbone → Neck (FPN+PAN) → Dual Detection Head.

### 3.2 Hệ thống Scale

YOLO26 hỗ trợ 5 kích thước qua hai hệ số depth (độ sâu) và width (độ rộng):

| Scale      | Depth | Width | Max Ch | Ghi chú                   |
|------------|-------|-------|--------|---------------------------|
| n (nano)   | 0.50  | 0.25  | 1024   | Nhỏ nhất, nhanh nhất      |
| s (small)  | 0.50  | 0.50  | 1024   | Cân bằng tốc độ/chính xác |
| m (medium) | 0.50  | 1.00  | 512    | Trung bình                |
| l (large)  | 1.00  | 1.00  | 512    | Lớn                       |
| x (xlarge) | 1.00  | 1.50  | 512    | Chính xác nhất            |

**Bảng 3.1:** Cấu hình Scale của YOLO26.

**Công thức tính kênh và lặp:**

$$\text{ch}(c) = \text{make\_divisible}(\min(c, \text{max\_ch}) \times \text{width}, 8) \quad (3.1)$$

$$\text{rep}(n) = \max(\text{round}(n \times \text{depth}), 1) \quad (3.2)$$

Ví dụ scale nano:  $\text{ch}(256) = \text{make\_divisible}(\min(256, 1024) \times 0.25, 8) = 64$ .

### 3.3 Backbone

Backbone gồm 11 layer, trích xuất đặc trưng từ ảnh RGB đầu vào.

| Layer | Module | Input             | Output (n)        | Key Args                      | Note         |
|-------|--------|-------------------|-------------------|-------------------------------|--------------|
| b0    | Conv   | $3 \times 320^2$  | $16 \times 160^2$ | $k=3, s=2$                    | P1/2         |
| b1    | Conv   | $16 \times 160^2$ | $32 \times 80^2$  | $k=3, s=2$                    | P2/4         |
| b2    | C3k2   | $32 \times 80^2$  | $64 \times 80^2$  | $\text{c3k}=\text{F}, e=0.25$ |              |
| b3    | Conv   | $64 \times 80^2$  | $64 \times 40^2$  | $k=3, s=2$                    | <b>P3/8</b>  |
| b4    | C3k2   | $64 \times 40^2$  | $128 \times 40^2$ | $\text{c3k}=\text{F}, e=0.25$ |              |
| b5    | Conv   | $128 \times 40^2$ | $128 \times 20^2$ | $k=3, s=2$                    | <b>P4/16</b> |
| b6    | C3k2   | $128 \times 20^2$ | $128 \times 20^2$ | $\text{c3k}=\text{T}$         |              |
| b7    | Conv   | $128 \times 20^2$ | $256 \times 10^2$ | $k=3, s=2$                    | <b>P5/32</b> |
| b8    | C3k2   | $256 \times 10^2$ | $256 \times 10^2$ | $\text{c3k}=\text{T}$         |              |
| b9    | SPPF   | $256 \times 10^2$ | $256 \times 10^2$ | $k=5, n=3$                    |              |
| b10   | C2PSA  | $256 \times 10^2$ | $256 \times 10^2$ | attention                     |              |

**Bảng 3.2:** Kiến trúc Backbone YOLO26n (scale nano, ảnh  $320 \times 320$ ).

**Các điểm quan trọng:**

- Layer b0–b1: Conv stride-2 giảm resolution 4x, tăng channels.
- Layer b2, b4: C3k2 với  $\text{c3k}=\text{False}$  (Bottleneck đơn giản) — feature đơn giản.
- Layer b6, b8: C3k2 với  $\text{c3k}=\text{True}$  (C3k kernel 3x3 sâu) — feature phức tạp.
- Layer b9: SPPF tạo multi-scale receptive field (5x5, 9x9, 13x13).

- Layer b10: C2PSA thêm self-attention ở P5 ( $10 \times 10$ ,  $N=100$ ,  $O(N^2)$  chấp nhận được).

## 3.4 Neck — FPN + PAN

Neck tổng hợp đặc trưng đa tỷ lệ bằng **Top-Down** (FPN) và **Bottom-Up** (PAN).

### 3.4.1 Top-Down Pathway (FPN)

Truyền semantic information từ tầng sâu (P5) xuống tầng nông (P3):

1. Upsample b10 ( $256 \times 10^2$ )  $\times 2 \rightarrow 256 \times 20^2$
2. Concat với b6 ( $128 \times 20^2$ )  $\rightarrow 384 \times 20^2$
3. C3k2  $\rightarrow$  h13:  $128 \times 20^2$
4. Upsample h13  $\times 2 \rightarrow 128 \times 40^2$
5. Concat với b4 ( $128 \times 40^2$ )  $\rightarrow 256 \times 40^2$
6. C3k2  $\rightarrow$  h16:  $64 \times 40^2$  (P3 output)

### 3.4.2 Bottom-Up Pathway (PAN)

Truyền detail information từ tầng nông (P3) lên tầng sâu (P5):

1. Conv stride-2: h16 ( $64 \times 40^2$ )  $\rightarrow 64 \times 20^2$
2. Concat với h13 ( $128 \times 20^2$ )  $\rightarrow 192 \times 20^2$
3. C3k2  $\rightarrow$  h19:  $128 \times 20^2$  (P4 output)
4. Conv stride-2: h19 ( $128 \times 20^2$ )  $\rightarrow 128 \times 10^2$
5. Concat với b10 ( $256 \times 10^2$ )  $\rightarrow 384 \times 10^2$
6. C3k2(attn=True)  $\rightarrow$  h22:  $256 \times 10^2$  (P5 output)

#### Tại sao cần cả hai chiều?

- **Top-Down** ( $\downarrow$ ): P3 nhận semantic từ P5  $\rightarrow$  phát hiện vật nhỏ với ngữ cảnh đầy đủ.
- **Bottom-Up** ( $\uparrow$ ): P5 nhận detail từ P3  $\rightarrow$  định vị chính xác hơn cho vật lớn.

### 3.4.3 Upsample — Nearest Neighbor

$$\text{Upsample}_{2\times} : [B, C, H, W] \rightarrow [B, C, 2H, 2W] \quad (3.3)$$

Mode nearest: mỗi pixel được nhân đôi theo cả 2 chiều. **Không có tham số học** (zero learnable params), nhanh.

### 3.4.4 Concat

Ghép nối theo chiều channel (dim=1):

$$\text{Concat} : [B, C_1, H, W] \oplus [B, C_2, H, W] \rightarrow [B, C_1 + C_2, H, W] \quad (3.4)$$

**Yêu cầu:**  $H, W$  phải bằng nhau. Đó là lý do phải Upsample trước khi Concat.

## 3.5 Detection Head — Dual Head

Detection Head nhận 3 feature maps (h16/P3, h19/P4, h22/P5) và tạo dự đoán.

### 3.5.1 Box Head (cv2)

$$\text{cv2} : \text{Conv}(x, c_2, 3) \rightarrow \text{Conv}(c_2, c_2, 3) \rightarrow \text{Conv2d}(c_2, 4 \times \text{reg\_max}, 1) \quad (3.5)$$

Với  $\text{reg\_max}=1$ : output = 4 channels ( $l, t, r, b$ ) trực tiếp.

### 3.5.2 Classification Head (cv3)

$$\text{cv3} : [\text{DWConv}(x) + \text{Conv}] \rightarrow [\text{DWConv} + \text{Conv}] \rightarrow \text{Conv2d}(c_3, nc, 1) \quad (3.6)$$

DWConv giảm tính toán so với Conv tiêu chuẩn.

### 3.5.3 One-to-Many (O2M) Head

Mỗi ground truth box gán cho **nhiều** anchor predictions (topk=10). Cung cấp dense supervision, **chỉ dùng khi training**.

### 3.5.4 One-to-One (O2O) Head

- Deep copy từ O2M: cùng kiến trúc, trọng số khởi tạo giống nhau.
- Mỗi GT chỉ gán cho **đúng 1** anchor tốt nhất (topk=7, topk2=1).
- Nhận **detached features** ( $x.\text{detach}()$ ) — gradient O2O không chảy về backbone.
- Dùng cho cả training và inference — **loại bỏ hoàn toàn NMS**.

### 3.5.5 Giải mã Bounding Box

Từ dự đoán  $(l, t, r, b)$  và anchor point  $(a_x, a_y)$ :

$$\text{bbox}_{\text{xyxy}} = (a_x - l, a_y - t, a_x + r, a_y + b) \times \text{stride} \quad (3.7)$$

### 3.5.6 Postprocessing (Inference)

1. Chỉ chạy O2O head (bỏ O2M)
2. Tính confidence:  $\text{score} = \sigma(\text{cls\_logits})$
3. Chọn Top-K predictions (mặc định  $K = 300$ )
4. Output:  $(x_1, y_1, x_2, y_2, \text{score}, \text{class\_id})$

### 3.5.7 Strip O2M khi export

Khi export cho inference, O2M head (cv2, cv3) bị loại bỏ:

| Metric          | Training (last.pt) | Inference (lite.pt) | Giảm |
|-----------------|--------------------|---------------------|------|
| File size       | 38.2 MB            | 5.6 MB              | ~85% |
| Precision       | FP32               | FP16                | 50%  |
| O2M Head        | Có                 | Stripped            | –    |
| Optimizer state | Có                 | Stripped            | –    |

**Bảng 3.3:** So sánh model trước và sau khi strip O2M.

## 3.6 Multi-Scale Detection

YOLO26 detect ở 3 scales, ảnh  $320 \times 320$ :

| Scale       | Stride | Feature Map    | Anchor Points | Phù hợp |
|-------------|--------|----------------|---------------|---------|
| P3          | 8      | $40 \times 40$ | 1600          | Vật nhỏ |
| P4          | 16     | $20 \times 20$ | 400           | Vật vừa |
| P5          | 32     | $10 \times 10$ | 100           | Vật lớn |
| <b>Tổng</b> |        |                | <b>2100</b>   |         |

**Bảng 3.4:** Số anchor points tại mỗi scale detection (ảnh  $320 \times 320$ ).

## CHƯƠNG 4

### Các khối Module chi tiết

#### 4.1 Conv — Convolution Block cơ bản

Khối Conv là đơn vị cơ bản nhất trong YOLO26:

$$\text{Conv}(x) = \text{SiLU}\left(\text{BatchNorm}(\text{Conv2d}(x))\right) \quad (4.1)$$

```
1 class Conv(nn.Module):
2     default_act = nn.SiLU()
3     def __init__(self, c1, c2, k=1, s=1, p=None, g=1, d=1, act=True):
4         self.conv = nn.Conv2d(c1, c2, k, s, autopad(k,p,d), groups=g,
5                               bias=False)
6         self.bn = nn.BatchNorm2d(c2)
7         self.act = self.default_act if act is True else (act or nn.
8                               Identity())
9     def forward(self, x):
10        return self.act(self.bn(self.conv(x)))
```

**Listing 4.1:** Conv block trong yolo26\_modules.py

**Conv 1x1** trộn channel tại cùng vị trí spatial. **Conv 3x3** trộn thông tin không gian (cạnh, biên, texture).

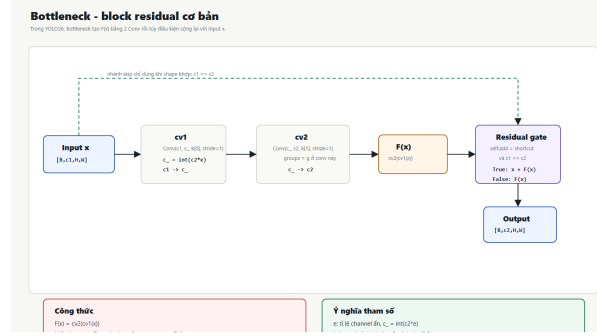
#### 4.2 DWConv — Depthwise Convolution

DWConv là Conv với  $\text{groups} = \text{gcd}(c_1, c_2)$ :

$$\text{Params}_{\text{DW}} = c_1 \times k^2 \ll \text{Params}_{\text{std}} = c_1 \times c_2 \times k^2 \quad (4.2)$$

Được dùng trong classification head (cv3) để giảm chi phí tính toán.

### 4.3 Bottleneck — Khối Residual cơ bản



Hình 4.1: Sơ đồ khối Bottleneck với skip connection.

$$\text{Bottleneck}(x) = \begin{cases} x + cv2(cv1(x)) & \text{nếu shortcut và } c_1 = c_2 \\ cv2(cv1(x)) & \text{ngược lại} \end{cases} \quad (4.3)$$

- cv1:  $\text{Conv}(c_1, c_-, k_1)$  — giảm kênh (squeeze) xuống  $c_- = \lfloor c_2 \times e \rfloor$
- cv2:  $\text{Conv}(c_-, c_2, k_2)$  — khôi phục kênh (expand)
- Skip connection: gradient lan truyền trực tiếp, giải quyết vanishing gradient

Ví dụ: input  $[B, 128, 80, 80]$ ,  $e = 0.5 \rightarrow c_- = 64 \rightarrow$  output  $[B, 128, 80, 80]$ . H, W giữ nguyên.

### 4.4 C2f — Cross Stage Partial with 2 Convolutions

C2f [16] kết hợp CSPNet với gradient flow hiệu quả:

$$\text{C2f}(x) = cv2\left(\text{Concat}\left[y_0, y_1, m_1(y_1), m_2(m_1(y_1)), \dots\right]\right) \quad (4.4)$$

$[y_0, y_1] = \text{chunk}(cv1(x), 2)$  chia đều ra thành 2 phần. Mỗi  $m_i$  là Bottleneck.

**Flow (n=2):**

```

Input -> cv1 -> split -> [y0, y1]
      |
      v
Bottleneck(y1) -> y2
      |
      v
Bottleneck(y2) -> y3
      |
      v
cat(y0, y1, y2, y3) -> cv2 -> Output
  
```

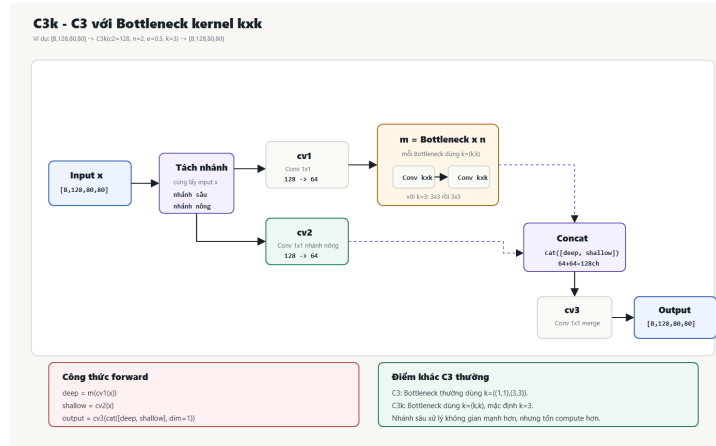
**Ý nghĩa:** Giữ lại feature nông ( $y_0$ ) lẫn feature sâu ( $y_3$ ), rich gradient flow.

## 4.5 C3 và C3k

**C3** là khối CSP kiểu cũ: hai nhánh — nhánh xử lý sâu (cv1 → bottleneck sequence) và nhánh đi thẳng (cv2) — rồi concat và merge (cv3).

**C3k** kế thừa C3 nhưng dùng Bottleneck với kernel  $k \times k$  (thường  $k = 3$ ):

$$\text{C3k} : \text{Bottleneck}_{\text{C3}} \text{ dùng } k = ((1, 1), (3, 3)) \rightarrow \text{C3k dùng } k = (3, 3) \quad (4.5)$$



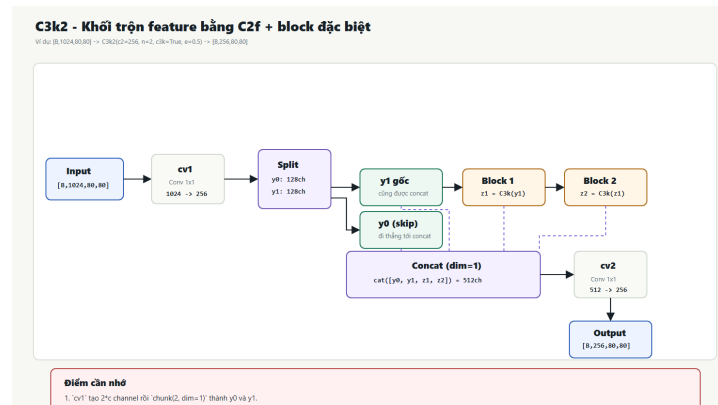
**Hình 4.2:** Sơ đồ khối C3k — C3 với kernel 3x3 sâu hơn.

C3k nhìn vùng lân cận tốt hơn, học hình dạng/biên tốt hơn, nhưng nặng hơn Bottleneck đơn giản.

## 4.6 C3k2 — Khối quan trọng nhất trong YOLO26

C3k2 kế thừa từ C2f nhưng thay danh sách block:

$$m_i = \begin{cases} \text{Bottleneck} + \text{PSABlock} & \text{nếu attn=True} \\ \text{C3k(kernel = 3)} & \text{nếu c3k=True} \\ \text{Bottleneck tiêu chuẩn} & \text{mặc định} \end{cases} \quad (4.6)$$



**Hình 4.3:** Sơ đồ khối C3k2 — forward theo C2f, block bên trong tùy cấu hình.

| Layer   | c3k   | attn  | Module bên trong      | Vị trí          |
|---------|-------|-------|-----------------------|-----------------|
| b2, b4  | False | False | Bottleneck đơn giản   | Backbone đầu    |
| b6, b8  | True  | False | C3k (3x3 sâu)         | Backbone P4, P5 |
| h13–h19 | True  | False | C3k                   | Neck            |
| h22     | True  | True  | Bottleneck + PSABlock | Neck P5 output  |

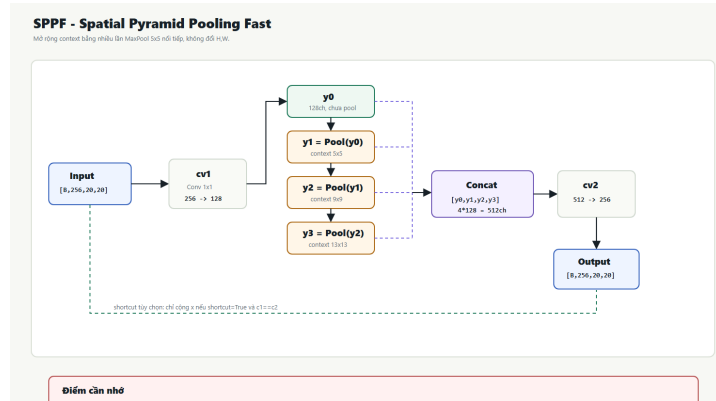
**Bảng 4.1:** Sử dụng C3k2 trong YOLO26.

**Ví dụ shape:** C3k2( $c_1 = 1024$ ,  $c_2 = 256$ ,  $n = 2$ ,  $c3k = \text{True}$ ,  $e = 0.5$ ):

[B, 1024, 80, 80] → cv1 → [B, 256, 80, 80]  
 → split: y0 [B, 128, 80, 80], y1 [B, 128, 80, 80]  
 → C3k(y1) → y2 [B, 128, 80, 80]  
 → C3k(y2) → y3 [B, 128, 80, 80]  
 → cat(y0,y1,y2,y3) → [B, 512, 80, 80]  
 → cv2 → [B, 256, 80, 80]

H, W giữ nguyên: tất cả conv dùng stride=1, padding autopad.

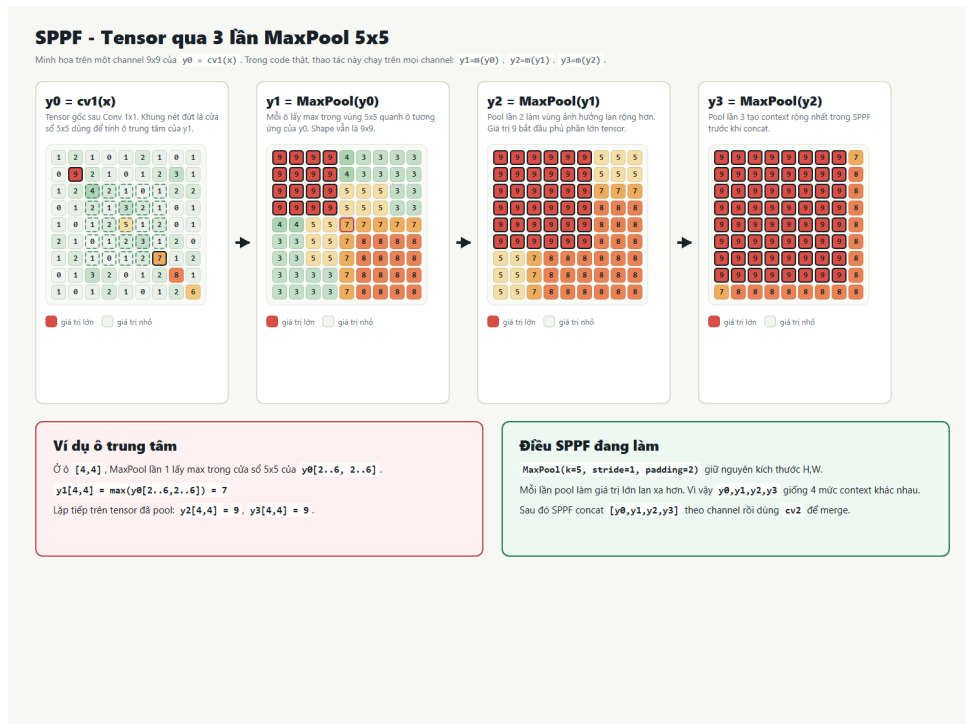
## 4.7 SPPF — Spatial Pyramid Pooling Fast



Hình 4.4: Sơ đồ khối SPPF — MaxPool liên tiếp tạo multi-scale receptive field.

$$\text{SPPF}(x) = \text{cv2}\left(\text{Concat}\left[y, \text{MP}(y), \text{MP}^2(y), \text{MP}^3(y)\right]\right) + x \quad (4.7)$$

$y = \text{cv1}(x)$ ,  $\text{MP} = \text{MaxPool2d}(k = 5, s = 1, p = 2)$ . Shortcut: output =  $\text{SPPF}(x) + x$ .



Hình 4.5: Minh họa MaxPool 5x5 qua 3 lần liên tiếp — tạo receptive field 5x5, 9x9, 13x13.

**Ý nghĩa:** MaxPool liên tiếp 3 lần tạo 3 mức receptive field, giúp model “nhìn” context ở nhiều phạm vi mà không giảm resolution.

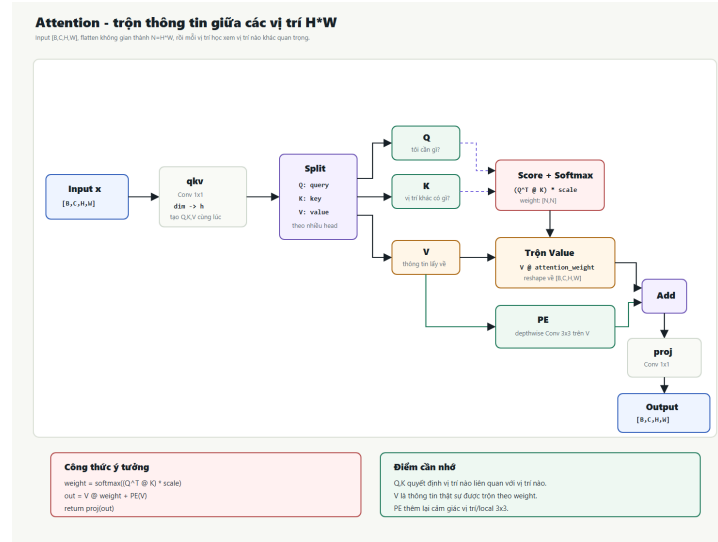
**MaxPool không có tham số học:** chỉ lấy giá trị lớn nhất trong vùng  $5 \times 5$ .

**So với SPP:** SPP dùng 3 kernel song song (5, 9, 13); SPPF dùng cùng kernel 5 nối tiếp — cho receptive field tương đương nhưng **nhANH hơn**.

**Lưu ý: YOLO26 SPPF có act=False trên cv1**

Khác với SPPF gốc (Ultralytics) dùng act=True (SiLU) trên cv1, YOLO26 dùng act=False. Điều này giảm representational power trước max-pooling.

## 4.8 Attention — Multi-Head Self-Attention



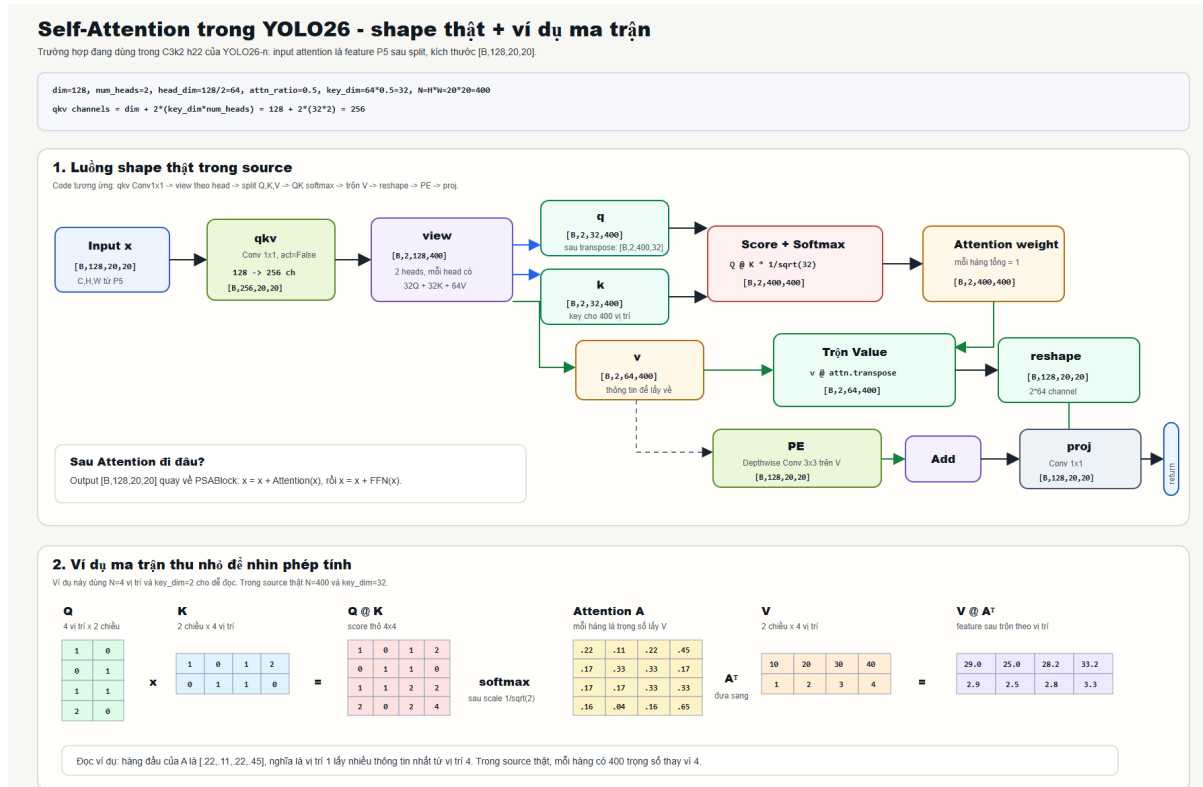
**Hình 4.6:** Sơ đồ khối Attention với QKV projection và Position Encoding.

$$Q, K, V = \text{split}(\text{QKV\_Conv}(x)) \quad (4.8)$$

$$\text{Attn} = \text{softmax}\left(\frac{Q^T K}{\sqrt{d_k}}\right) \quad (4.9)$$

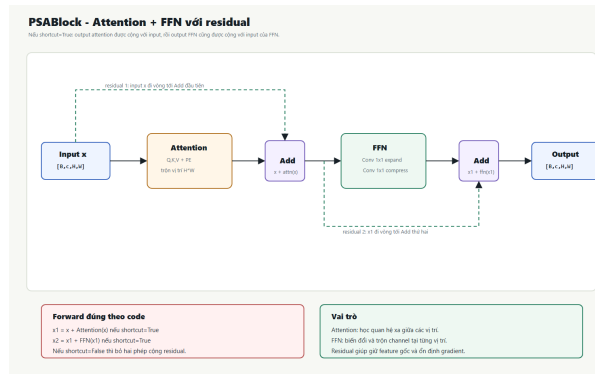
$$\text{output} = V \cdot \text{Attn}^T + \text{PE}(V) \quad (4.10)$$

- **QKV Conv:** Conv  $1 \times 1$  tạo đồng thời  $Q, K, V$ . Không activation.
- $d_k = \text{head\_dim} \times \text{attn\_ratio}$ : chiều Key giảm để tiết kiệm.
- **PE:** DWConv  $3 \times 3$  trên  $V$  mã hóa vị trí spatial.
- Chia  $h$  heads, mỗi head xử lý  $C/h$  kênh.



Hình 4.7: Chi tiết flow và shape biến đổi trong Attention module.

## 4.9 PSABlock — Positional Self-Attention Block



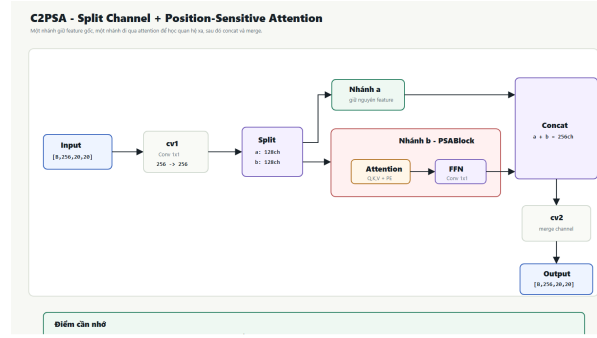
Hình 4.8: Sơ đồ PSABlock: Attention + FFN với residual connections.

$$x_1 = x + \text{Attention}(x) \quad (\text{residual attention}) \quad (4.11)$$

$$x_2 = x_1 + \text{FFN}(x_1) \quad (\text{residual FFN}) \quad (4.12)$$

FFN: Conv  $1 \times 1$  ( $c \rightarrow 2c$ )  $\rightarrow$  Conv  $1 \times 1$  ( $2c \rightarrow c$ , act=False).

## 4.10 C2PSA — Channel-to-Positional Self-Attention



Hình 4.9: Sơ đồ C2PSA: CSP splitting + PSABlock.

$$a, b = \text{split}(\text{cv1}(x)) \quad (4.13)$$

$$b' = \text{PSABlock}(b) \quad (4.14)$$

$$\text{output} = \text{cv2}(\text{Concat}[a, b']) \quad (4.15)$$

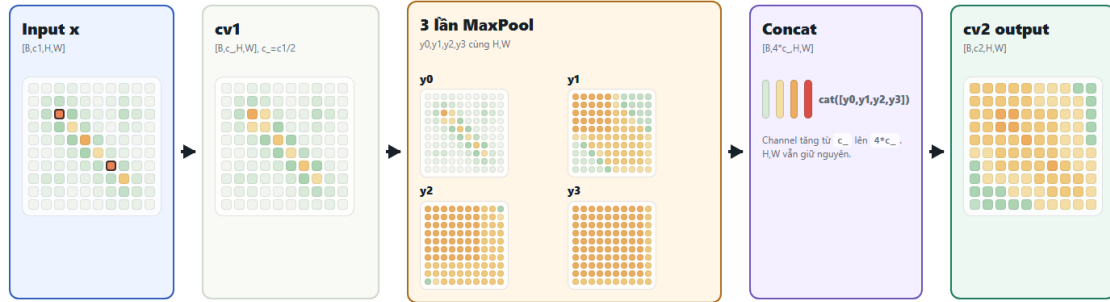
Chỉ nhánh  $b$  qua attention (tồn kém), nhánh  $a$  giữ nguyên. C2PSA chỉ dùng ở **tầng b10** (stride 32) vì  $10 \times 10$ ,  $N = 100$  — chi phí  $O(N^2)$  chấp nhận được.

### Feature map biến đổi qua SPPF và C2PSA

Heatmap dưới đây là dữ liệu minh họa, không phải activation thật của model. Mục tiêu là nhìn được thao tác tensor: SPPF lan context bằng MaxPool nhiều lần, còn C2PSA tách channel rồi cho một nhánh học quan hệ xa bằng attention.

#### 1. SPPF: giữ H,W, mở rộng context, concat nhiều mức

Trong code: `y0=cv1(x)` , `y1=MaxPool(y0)` , `y2=MaxPool(y1)` , `y3=MaxPool(y2)` , rồi `cv2(cat([y0,y1,y2,y3]))` .



##### Nhìn trên H,W

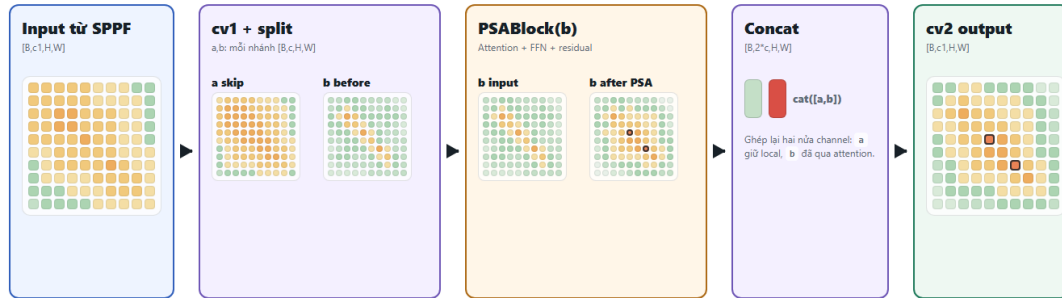
MaxPool 5x5 không đổi kích thước, nhưng vùng activation mạnh lan rộng ra sau mỗi lần pool. Vì vậy một vị trí sau SPPF mang thêm context từ vùng lân cận rộng hơn.

##### Nhìn trên channel

cv1 giảm channel về  $c=c1/2$ . Sau concat, channel thành  $4*c$ . cv2 trộn lại các mức context để ra  $c2$  channel.

#### 2. C2PSA: giữ H,W, split channel, attention một nhánh

Trong code: `a,b = cv1(x).split(...)` , `b = PSABlock(b)` , rồi `cv2(cat([a,b]))` .



##### Nhánh a

Nhánh a giữ feature gốc hơn. Nó giúp block không làm mất thông tin CNN/local đã có.

##### Nhánh b

Nhánh b đi qua PSABlock. Attention làm các vùng xa nhau nhưng liên quan cùng tăng ảnh hưởng lên nhau.

Hình 4.10: Biến đổi feature map qua SPPF và C2PSA.

### Tại sao chỉ dùng Attention ở P5?

Self-attention có complexity  $O(N^2)$  với  $N = H \times W$ . Ở P5 ( $10 \times 10$ ),  $N = 100$  — rất nhẹ. Ở P3 ( $40 \times 40$ ),  $N = 1600$  — quá nặng cho model Nano. Tương tự, h22 dùng C3k2 với `attn=True` cũng ở scale P5.

## 4.11 DFL — Distribution Focal Loss Module

DFL [17] biến regression thành classification trên `reg_max` bins rồi rạc:

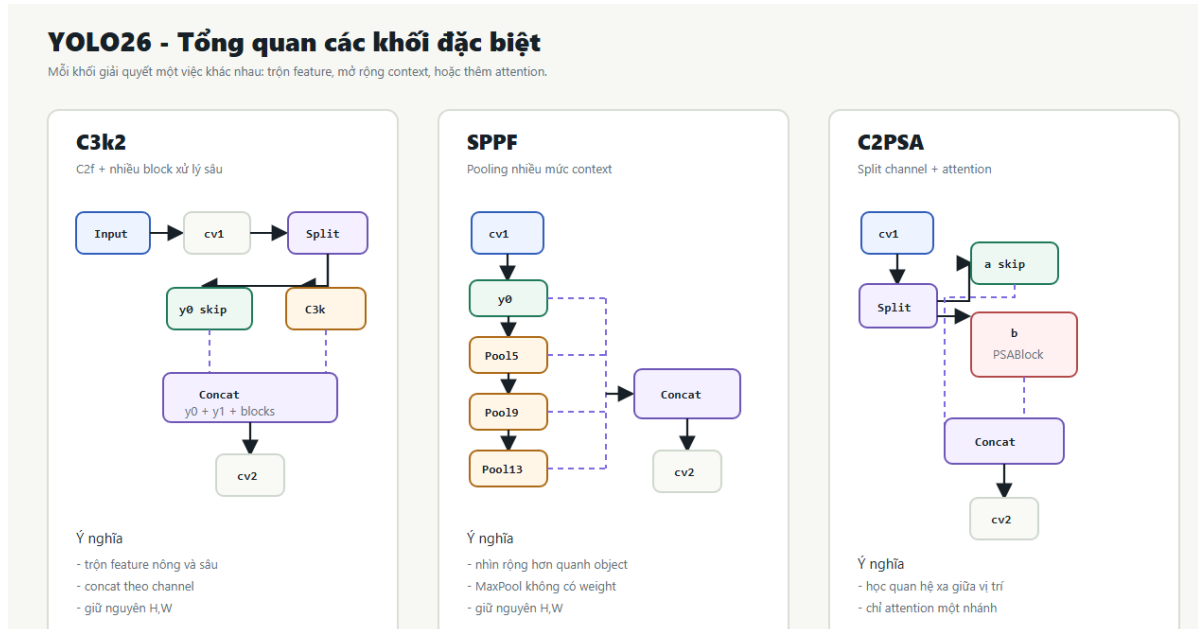
$$\hat{d} = \sum_{i=0}^{\text{reg\_max}-1} i \cdot p_i, \quad p_i = \text{softmax}(\text{logits})_i \quad (4.16)$$

Weight của Conv  $1 \times 1$  cố định là  $[0, 1, 2, \dots, \text{reg\_max} - 1]$ .

#### YOLO26 mặc định: reg\_max=1 — không dùng DFL

Khi reg\_max=1, DFL được thay bằng nn.Identity(). Model dự đoán trực tiếp 4 giá trị  $(l, t, r, b)$ , output box head = 4 channels. Với reg\_max=16: 64 channels, chính xác hơn nhưng nặng hơn.

## 4.12 Tổng quan đồ thị module



**Hình 4.11:** Tổng quan các khối đặc biệt: C3k2, SPPF, C2PSA trong kiến trúc YOLO26.

## CHƯƠNG 5

# Hàm mất mát

### 5.1 Tổng quan Loss trong YOLO26

Tổng loss gồm 3 thành phần:

$$\mathcal{L}_{\text{head}} = w_{\text{box}} \cdot \mathcal{L}_{\text{box}} + w_{\text{cls}} \cdot \mathcal{L}_{\text{cls}} + w_{\text{dfl}} \cdot \mathcal{L}_{\text{dfl}} \quad (5.1)$$

| Thành phần                 | Source                   | $w$ đầu | $w$ cuối | Mục đích                   |
|----------------------------|--------------------------|---------|----------|----------------------------|
| $\mathcal{L}_{\text{box}}$ | cv2 $\rightarrow$ CIoU   | 7.5     | 3.75     | Định vị (localization)     |
| $\mathcal{L}_{\text{cls}}$ | cv3 $\rightarrow$ BCE    | 0.5     | 1.5      | Phân loại (classification) |
| $\mathcal{L}_{\text{dfl}}$ | cv2 $\rightarrow$ DFL/L1 | 1.5     | 1.5      | Phân phối khoảng cách      |

**Bảng 5.1:** Trọng số các thành phần loss (thay đổi theo ProgLoss).

### 5.2 Box Loss — CIoU Loss

$$\mathcal{L}_{\text{box}} = \frac{\sum_{i \in \text{fg}} (1 - \text{CIoU}_i) \cdot w_i}{\sum_j t_j} \quad (5.2)$$

$w_i = \sum_c t_{i,c}$  là tổng target scores của anchor  $i$  (chỉ foreground),  $t_j = \text{target\_scores\_sum}$ .

**CIoU** (xem Mục 2.2.3) xét cả khoảng cách tâm, diện tích giao, và tỷ lệ khung hình — gradient tốt hơn IoU thuần.

### 5.3 Classification Loss — BCE Loss

$$\mathcal{L}_{\text{cls}} = \frac{\sum_{i,c} \text{BCE}(\hat{s}_{i,c}, t_{i,c})}{\sum_j t_j} \quad (5.3)$$

Target scores  $t_{i,c}$  là **soft labels** từ alignment metric (không phải 0/1 cứng), giúp model học “mức độ phù hợp” thay vì chỉ binary.

### 5.4 DFL/LTRB Loss

#### 5.4.1 Khi $\text{reg\_max} > 1$ : DFL Loss

$$\mathcal{L}_{\text{DFL}}(d) = -\left[ (d_{\top} - d) \log p_{d_{\top}} + (d - d_{\text{L}}) \log p_{d_{\text{L}}} \right] \quad (5.4)$$

$d$  = target liên tục,  $d_{\downarrow} = \lfloor d \rfloor$ ,  $d_{\uparrow} = d_{\downarrow} + 1$ ,  $p_i = \text{softmax}(\text{logits})_i$ .

### 5.4.2 Khi `reg_max = 1`: L1 Loss (YOLO26 default)

$$\mathcal{L}_{L1} = \frac{\sum_{i \in \text{fg}} \|p_i - t_i\|_1 \cdot w_i}{\sum_j t_j} \quad (5.5)$$

Trong đó  $p_i, t_i$  là predicted/target LTRB distances, được normalize theo image size:

$$t_{\text{ltrb}}[0 :: 2] \text{ /= width, } t_{\text{ltrb}}[1 :: 2] \text{ /= height} \quad (5.6)$$

## 5.5 TaskAlignedAssigner (TAL)

TAL xác định anchor nào là positive/negative cho mỗi GT box.

### 5.5.1 Alignment Metric

$$\text{align\_metric}_{i,j} = s_{i,j}^{\alpha} \cdot u_{i,j}^{\beta} \quad (5.7)$$

- $s_{i,j}$ : classification score (sigmoid) của anchor  $i$  cho class GT  $j$
- $u_{i,j}$ : CIoU giữa predicted box và GT box
- $\alpha = 0.5, \beta = 6.0$ : CIoU được nhấn mạnh rất mạnh ( $\beta = 6$ )

### 5.5.2 Quy trình gán nhãn

---

#### Thuật toán 1 TaskAlignedAssigner

---

- 1: **Input**: `pred_scores`, `pred_bboxes`, `anchor_points`, `gt_labels`, `gt_bboxes`
  - 2: **Step 1**: Lọc anchors nằm trong GT box (geometric prior)
  - 3: **Step 2**: Tính `alignment_metric = s0.5 · CIoU6`
  - 4: **Step 3**: Chọn Top-K anchors có metric cao nhất cho mỗi GT  
O2M:  $K = 10$ , O2O:  $K = 7 \rightarrow$  chọn Top-1
  - 5: **Step 4**: Xử lý conflict: nếu anchor match nhiều GT, giữ GT có IoU cao nhất
  - 6: **Step 5**: Tính `soft labels = one-hot × normalized metric`
  - 7: **Output**: `target_labels`, `target_bboxes`, `target_scores`, `fg_mask`
- 

#### Anchor gần tâm không chắc được chọn

Vì  $\beta = 6.0$ , CIoU ảnh hưởng rất mạnh. Anchor lệch tâm nhưng có `pred_box` khớp GT hơn và class score tốt hơn có thể được ưu tiên hơn anchor ngay tâm.

### 5.5.3 STAL — Small-Target-Aware Label Assignment

Đối tượng nhỏ hơn  $\text{stride}[0]$  pixel có nguy cơ không match anchor nào. STAL phóng to bbox:

$$\text{bbox}'_{w,h} = \begin{cases} \text{stride}[1] & \text{nếu } w \text{ hoặc } h < \text{stride}[0] \\ w, h & \text{ngược lại} \end{cases} \quad (5.8)$$

## 5.6 E2ELoss — End-to-End Loss

E2ELoss kết hợp loss từ cả O2M và O2O head:

$$\mathcal{L}_{\text{E2E}} = \alpha(e) \cdot \mathcal{L}_{\text{O2M}} + \beta(e) \cdot \mathcal{L}_{\text{O2O}}, \quad \alpha + \beta = 1 \quad (5.9)$$

Trọng số thay đổi theo epoch:

$$\alpha(e) = \max \left( 1 - \frac{e}{\max(E-1, 1)}, 0 \right) \times (\alpha_0 - \alpha_f) + \alpha_f \quad (5.10)$$

$$\beta(e) = 1 - \alpha(e) \quad (5.11)$$

$$\alpha_0 = 0.8, \alpha_f = 0.3.$$

| Epoch | $\alpha$ (O2M) | $\beta$ (O2O) | Ý nghĩa                              |
|-------|----------------|---------------|--------------------------------------|
| 1     | 0.80           | 0.20          | O2M ưu thế — dense supervision       |
| 50    | 0.55           | 0.45          | Cân bằng dần                         |
| 100   | 0.30           | 0.70          | O2O tiếp quản — chuyên cho inference |

**Bảng 5.2:** Lịch trình trọng số O2M/O2O qua epoch.

## 5.7 ProgLoss — Progressive Loss Balancing

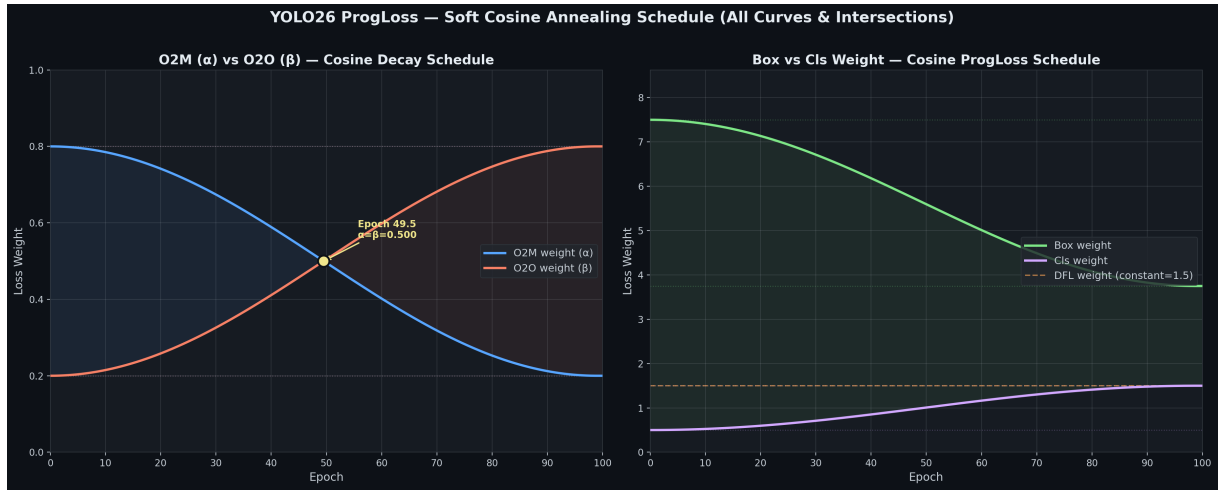
ProgLoss thay đổi trọng số box/cls loss theo thời gian huấn luyện:

$$t = \min \left( \frac{\text{epoch}}{\text{total\_epochs} - 1}, 1 \right) \quad (5.12)$$

$$\text{progress} = \frac{1}{2} (1 - \cos(\pi \cdot t)) \quad (\text{cosine annealing}) \quad (5.13)$$

$$w_{\text{box}}(e) = 7.5 + (3.75 - 7.5) \times \text{progress} \quad (5.14)$$

$$w_{\text{cls}}(e) = 0.5 + (1.5 - 0.5) \times \text{progress} \quad (5.15)$$



**Hình 5.1:** ProgLoss curves. Trái: O2M/O2O weights. Phải: Box weight giảm, Cls weight tăng.

**Triết lý:** Giai đoạn đầu tập trung học ở đâu (box weight cao), giai đoạn sau tập trung học là gì (cls weight cao).

**Cosine vs Linear:** Cosine gần như không đổi ở 20 epoch đầu (ổn định), thay đổi mạnh ở giữa, nhẹ nhàng hội tụ cuối. Linear thay đổi đều, có thể gây gradient shock.

## 5.8 Công thức ghép đầy đủ

$$\mathcal{L}_{\text{total}}(e) = \alpha(e) \cdot \left[ w_{\text{box}}(e) \cdot \mathcal{L}_{\text{box}}^{\text{O2M}} + w_{\text{cls}}(e) \cdot \mathcal{L}_{\text{cls}}^{\text{O2M}} + w_{\text{dfl}} \cdot \mathcal{L}_{\text{dfl}}^{\text{O2M}} \right] + \beta(e) \cdot \left[ w_{\text{box}}(e) \cdot \mathcal{L}_{\text{box}}^{\text{O2O}} + w_{\text{cls}}(e) \cdot \mathcal{L}_{\text{cls}}^{\text{O2O}} \right] \quad (5.16)$$

**Trực giác:**

- Đầu training:  $\alpha$  cao  $\rightarrow$  nhiều anchor positive (dense supervision);  $w_{\text{box}}$  cao  $\rightarrow$  ưu tiên localization.
- Cuối training:  $\beta$  cao  $\rightarrow$  O2O head quan trọng hơn;  $w_{\text{cls}}$  cao  $\rightarrow$  tập trung classification.

## CHƯƠNG 6

# Chiến lược huấn luyện

### 6.1 Training Loop

Training loop chuẩn PyTorch:

```
for epoch in range(epochs):
    scheduler.step()
    model.train()
    for batch in train_loader:
        imgs = batch["img"].cuda().float() / 255
        with torch.autocast("cuda"):
            preds = model(imgs)
            loss = criterion(preds, batch)
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        clip_grad_norm_(model.parameters(), 10.0)
        scaler.step(optimizer); scaler.update()
        optimizer.zero_grad()
        ema.update(model)
    criterion.update() # ProgLoss
    metrics = validate(ema.ema, val_loader)
    save_model(epoch, ema, optimizer, metrics)
    if stopper(epoch, fitness): break
```

### 6.2 Optimizer — Phân nhóm tham số

YOLO26 chia tham số thành 3 nhóm:

| Nhóm     | Loại                       | Cấu hình                              |
|----------|----------------------------|---------------------------------------|
| g_bn     | BatchNorm weights          | SGD, no weight decay                  |
| g_weight | Conv weights ( $\geq 2D$ ) | SGD + weight decay $5 \times 10^{-4}$ |
| g_bias   | Bias                       | SGD, no weight decay                  |

**Bảng 6.1:** Phân nhóm tham số optimizer.

**Hyperparameters:**  $lr_0 = 0.01$ , momentum = 0.937, weight\_decay =  $5 \times 10^{-4}$ , Nesterov = True.

## 6.3 Learning Rate Schedule

Cosine annealing LR scheduler:

$$\text{lr}(e) = \text{lr}_0 \times \left[ \frac{1 - \cos(\pi \cdot e/E)}{2} \times (\text{lrf} - 1) + 1 \right] \quad (6.1)$$

$\text{lrf} = 0.01$  (tỷ lệ LR cuối/đầu). LR giảm từ 0.01 xuống 0.0001 theo đường cosine.

## 6.4 Warmup

Trong  $n_w = \max(3 \times n_b, 100)$  iterations đầu:

$$\text{lr} : 0 \rightarrow \text{lr}_0 \times \text{lf}(\text{epoch}) \quad (6.2)$$

$$\text{momentum} : 0.8 \rightarrow 0.937 \quad (6.3)$$

Tăng tuyến tính bằng `np.interp()`.

## 6.5 Exponential Moving Average (EMA)

### Khái niệm: EMA

EMA duy trì bản sao “trung bình” của model weights:

$$\theta_{\text{EMA}}^{(t)} = d(t) \cdot \theta_{\text{EMA}}^{(t-1)} + (1 - d(t)) \cdot \theta_{\text{model}}^{(t)} \quad (6.4)$$

$$d(t) = d_0 \times \left( 1 - e^{-t/\tau} \right), \quad d_0 = 0.9999, \tau = 2000 \quad (6.5)$$

EMA cho kết quả ổn định hơn, được dùng cho validation và lưu checkpoint.

## 6.6 Automatic Mixed Precision (AMP)

AMP sử dụng FP16 cho forward pass (tiết kiệm bộ nhớ, tăng tốc), FP32 cho gradient accumulation (ổn định số học):

- `torch.autocast("cuda")`: Tự chọn FP16/FP32 cho từng operation
- `GradScaler`: Scale loss trước backward, unscale gradient trước optimizer step

## 6.7 Gradient Clipping

Giới hạn norm gradient để tránh exploding gradient:

$$\text{clip\_grad\_norm\_}(\theta, \text{max\_norm} = 10.0) \quad (6.6)$$

## 6.8 Gradient Accumulation

Khi batch size nhỏ, gradient tích lũy qua nhiều bước:

$$\text{accumulate} = \max \left( \text{round} \left( \frac{64}{\text{batch\_size}} \right), 1 \right) \quad (6.7)$$

## 6.9 Data Augmentation

| Augmentation             | Tham số                                              | P   |
|--------------------------|------------------------------------------------------|-----|
| Random Scale + Translate | scale $\in [0.5, 1.5]$ , translate $\in [-0.1, 0.1]$ | 0.5 |
| Horizontal Flip          | —                                                    | 0.5 |
| HSV Augmentation         | H: 0.015, S: 0.7, V: 0.4                             | 0.5 |
| Random Erasing           | 5–20% diện tích                                      | 0.3 |

**Bảng 6.2:** Data augmentation trong YOLO26.

### Thiếu Mosaic augmentation

YOLO26 hiện chưa triển khai Mosaic (ghép 4 ảnh), MixUp, và Copy-Paste — những augmentation mạnh nhất cho YOLO. Đây là nguyên nhân chính gây overfitting khi train lâu.

## 6.10 EarlyStopping

Dừng training khi fitness không cải thiện sau patience epochs:

$$\text{stop} = (\text{epoch} - \text{best\_epoch}) \geq \text{patience} \quad (6.8)$$

Mặc định: patience = 50.

## 6.11 Letterbox Resize

Ảnh được resize về kích thước cố định ( $320 \times 320$  hoặc  $640 \times 640$ ) bằng letterbox:

1. Tính tỷ lệ scale sao cho ảnh fit trong target size
2. Resize theo tỷ lệ đó (giữ nguyên aspect ratio)
3. Pad phần dư bằng pixel (114, 114, 114)

## CHƯƠNG 7

### Muon Optimizer và MuSGD

#### 7.1 Vấn đề của SGD/AdamW truyền thống

SGD và AdamW cập nhật trọng số theo từng phần tử (element-wise), bỏ qua cấu trúc hình học của ma trận trọng số. Khi các hướng (principal components) có gradient chênh lệch lớn (ill-conditioned), bước cập nhật bị mất cân bằng nghiêm trọng.

##### Vấn đề Ill-Conditioning

Xét ma trận gradient  $G = \begin{bmatrix} 2 & -1 \\ 2 & 1 \end{bmatrix}$  và  $W = I, \eta = 0.1$ :

$$W_{\text{SGD}} = W - \eta G = \begin{bmatrix} 0.8 & 0.1 \\ -0.2 & 0.9 \end{bmatrix} \quad (7.1)$$

Ma trận kết quả bị lệch trục nghiêm trọng do gradient không cân bằng giữa các chiều.

#### 7.2 Giải pháp: Trục giao hóa Gradient

Muon [18] trục giao hóa ma trận gradient: bóc tách và loại bỏ scale ( $\Sigma$ ), chỉ giữ ma trận hướng trục giao  $O = UV^T$ .

$$W_{t+1} = W_t - \eta \cdot O, \quad \text{trong đó } O = UV^T \text{ với } G = U\Sigma V^T \quad (7.2)$$

Kết quả: mọi singular values của  $O$  đều bằng 1, đảm bảo bước cập nhật đều đặn trên mọi đặc trưng.

$$W_{\text{Muon}} = I - 0.1 \times \begin{bmatrix} 0.7071 & -0.7071 \\ 0.7071 & 0.7071 \end{bmatrix} = \begin{bmatrix} 0.929 & 0.071 \\ -0.071 & 0.929 \end{bmatrix} \quad (7.3)$$

Cân bằng hoàn hảo so với SGD.

#### 7.3 Singular Value Decomposition (SVD)

##### Khái niệm: SVD

Phân rã giá trị kỳ dị:  $A = U\Sigma V^T$

- $V^T$ : Xoay không gian đầu vào (ma trận trực giao)
- $\Sigma$ : Co giãn theo các giá trị kỳ dị  $\sigma_i$  (ma trận đường chéo)
- $U$ : Xoay đến không gian đầu ra (ma trận trực giao)

**Cách tìm SVD từ  $M = A^T A$ :**

1. Tìm trị riêng:  $\det(M - \lambda I) = 0$
2. Giá trị kỳ dị:  $\sigma_i = \sqrt{\lambda_i}$
3. Vector riêng:  $(M - \lambda_i I)v_i = 0$ , chuẩn hóa  $\|v_i\| = 1$
4. Ma trận trực giao:  $O = UV^T$

**Hạn chế:** SVD có complexity  $O(mn^2)$ , sử dụng Householder reflections — sequential, CPU-bound, không phù hợp GPU.

## 7.4 Newton–Schulz Iteration

Newton–Schulz xấp xỉ  $O = UV^T$  chỉ bằng phép nhân ma trận (GPU-friendly).

### 7.4.1 Bước 0: Chuẩn hóa Frobenius

$$X_0 = \frac{G}{\|G\|_F}, \quad \|G\|_F = \sqrt{\sum_{i,j} g_{ij}^2} = \sqrt{\sum_i \sigma_i^2} \quad (7.4)$$

Đảm bảo tổng bình phương singular values bằng 1, suy ra mọi  $\sigma \leq 1$  (vùng hội tụ an toàn).

### 7.4.2 Công thức lặp (Standard — Quadratic)

$$X_{k+1} = \frac{3}{2}X_k - \frac{1}{2}X_k(X_k^T X_k) \quad (7.5)$$

### 7.4.3 Dạng Cubic (dùng trong Muon/YOLO26)

$$A_k = X_k X_k^T \quad (7.6)$$

$$X_{k+1} = a X_k + b (A_k X_k) + c (A_k^2 X_k) \quad (7.7)$$

Hệ số tối ưu cho cubic convergence:  $a = 3.4445$ ,  $b = -4.7750$ ,  $c = 2.0315$ .

### 7.4.4 Ý nghĩa đa thức

Newton–Schulz áp dụng đa thức  $f(\sigma)$  lên các singular values:

$$f(\sigma) = \frac{3}{2}\sigma - \frac{1}{2}\sigma^3 \quad (\text{standard}) \quad (7.8)$$

$$f(\sigma) = a\sigma + b\sigma^3 + c\sigma^5 \quad (\text{cubic}) \quad (7.9)$$

Đa thức này hoạt động như “lực hút” ép mọi  $\sigma$  về 1.0:

$$\Sigma = \text{diag}(\sigma_1, \dots, \sigma_n) \xrightarrow{f^{(T)}} I \quad (7.10)$$

## 7.5 Ví dụ tính toán chi tiết

**Ma trận gradient:**  $G = \begin{bmatrix} 2 & -1 \\ 2 & 1 \end{bmatrix}$

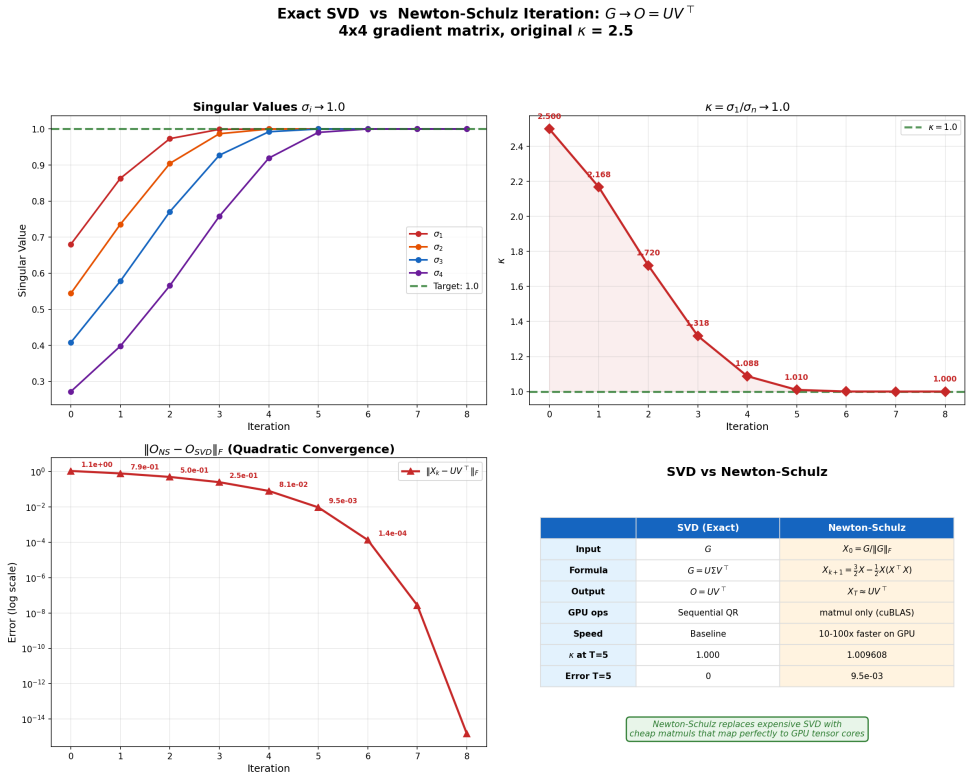
**Chuẩn Frobenius:**  $\|G\|_F = \sqrt{4 + 1 + 4 + 1} = \sqrt{10} \approx 3.162$

**Khởi tạo:**  $X_0 = \frac{1}{3.162} \begin{bmatrix} 2 & -1 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 0.632 & -0.316 \\ 0.632 & 0.316 \end{bmatrix}$

| Iter | $\sigma_1$ | $\sigma_2$ | $\kappa = \sigma_1/\sigma_2$ |
|------|------------|------------|------------------------------|
| 0    | 0.894      | 0.447      | 2.000                        |
| 1    | 0.866      | 0.625      | 1.387                        |
| 2    | 0.943      | 0.755      | 1.248                        |
| 3    | 0.992      | 0.916      | 1.083                        |
| 4    | 1.000      | 0.988      | 1.011                        |
| 5    | 1.000      | 1.000      | 1.000                        |

**Bảng 7.1:** Hội tụ singular values qua các bước Newton–Schulz.

Sau 5 bước:  $X_5 \approx \begin{bmatrix} 0.707 & -0.707 \\ 0.707 & 0.707 \end{bmatrix} = O_{\text{SVD}}$ , khớp 99.99%.



Hình 7.1: So sánh SVD exact vs Newton–Schulz: hội tụ singular values về 1.

7.6 Tốc độ hội tụ

Standard NS (Quadratic) :  $\|X_{k+1} - UV^T\| = O\left(\|X_k - UV^T\|^2\right)$  (7.11)

Cubic NS (Muon) :  $\|X_{k+1} - UV^T\| = O\left(\|X_k - UV^T\|^3\right)$  (7.12)

Ví dụ: error 0.1 → 0.001 → 10<sup>−9</sup> → 10<sup>−27</sup> (cubic) vs 0.1 → 0.01 → 10<sup>−4</sup> (quadratic).

7.7 MuSGD — Hybrid Optimizer trong YOLO26

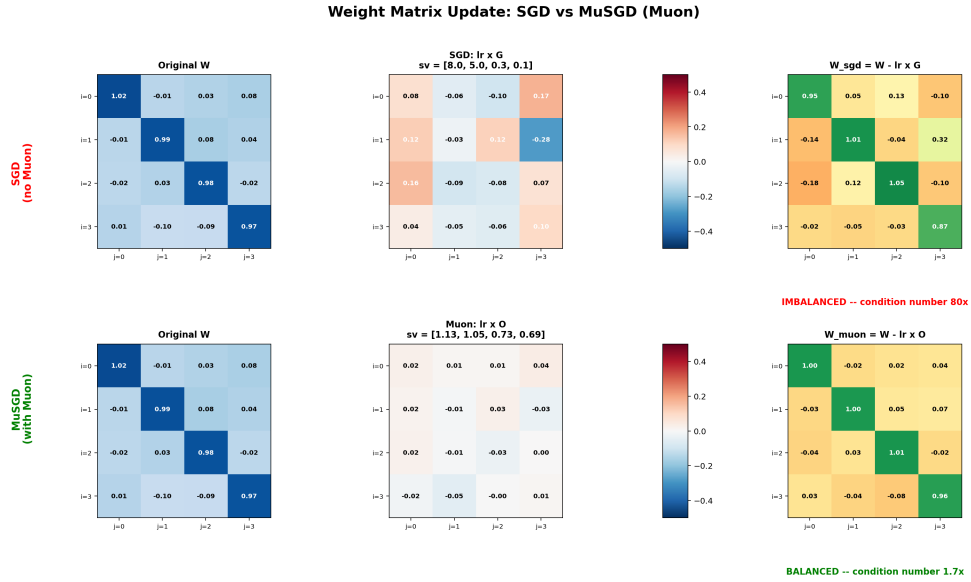
MuSGD kết hợp SGD truyền thống với Muon:

| Tham số             | Optimizer           | Lý do                                      |
|---------------------|---------------------|--------------------------------------------|
| BatchNorm weights   | SGD                 | 1D, không cần trực giao hóa                |
| Conv weights (≥ 2D) | SGD + Newton–Schulz | Mã trận 2D, hưởng lợi từ orthogonalization |
| Bias                | SGD                 | 1D vector                                  |

Bảng 7.2: Phân nhóm tham số trong MuSGD.

### Muon chỉ dùng cho ma trận 2D

Muon không hiệu quả trên vector 1D (bias, LayerNorm) do hiện tượng nhảy lố (oscillation) khi không có gradient thu hẹp. Chỉ áp dụng cho Linear, Attention, Conv weights.



Hình 7.2: So sánh hiệu ứng SGD vs Muon trên cập nhật ma trận trọng số.

## 7.8 Tóm tắt quy trình Muon

$$\underbrace{G}_{\text{gradient}} \xrightarrow{\text{normalize}} X_0 = \frac{G}{\|G\|_F} \xrightarrow[\text{T=5 iterations}]{\text{Newton-Schulz}} X_T \approx UV^T = O \quad (7.13)$$

$$W_{t+1} = W_t - \eta \cdot O \quad (\text{mọi } \sigma_i(O) = 1, \kappa = 1) \quad (7.14)$$

## Finding Orthogonal Update $O = UV^T$ via SVD

Complete step-by-step with a 2x2 gradient matrix

### STEP 0 Define Gradient Matrix $G$

$$G = \begin{bmatrix} 2 & -1 \\ 2 & 1 \end{bmatrix}$$

Raw gradient from backprop.  
Goal: find  $O = UV^T$

### STEP 1 Compute $M = G^T G$

$$G^T \begin{bmatrix} 2 & 2 \\ -1 & 1 \end{bmatrix} \times G \begin{bmatrix} 2 & -1 \\ 2 & 1 \end{bmatrix} = \begin{matrix} 2(2)+2(2)=8 & 2(-1)+2(1)=0 \\ (-1)(2)+1(2)=0 & (-1)(-1)+1(1)=2 \end{matrix}$$

$$M = \begin{bmatrix} 8 & 0 \\ 0 & 2 \end{bmatrix}$$

Diagonal matrix!  
Eigenvalues visible directly.

### STEP 2 Eigenvalues $\lambda$ from $\det(M - \lambda I) = 0$

$$\det(M - \lambda I) = (8 - \lambda)(2 - \lambda) = 0$$

$$\lambda_1 = 8 \qquad \lambda_2 = 2$$

### STEP 3 Singular Values $\sigma_i = \sqrt{\lambda_i}$

$$\sigma_1 = \sqrt{8} = 2\sqrt{2} \approx 2.8284 \qquad \sigma_2 = \sqrt{2} \approx 1.4142$$

$$\Sigma = \begin{bmatrix} 2.83 & 0.00 \\ 0.00 & 1.41 \end{bmatrix}$$

Condition number:  $\kappa = 2.83/1.41 = 2.0$

### STEP 4 Eigenvectors $(M - \lambda_i I)v = 0 \rightarrow$ columns of $V$

$$\lambda_1 = 8: \quad (M - 8I)v = 0$$

$$\begin{bmatrix} 0 & 0 \\ 0 & -6 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 0 \qquad -6v_2 = 0 \Rightarrow v_2 = 0 \qquad v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\lambda_2 = 2: \quad (M - 2I)v = 0$$

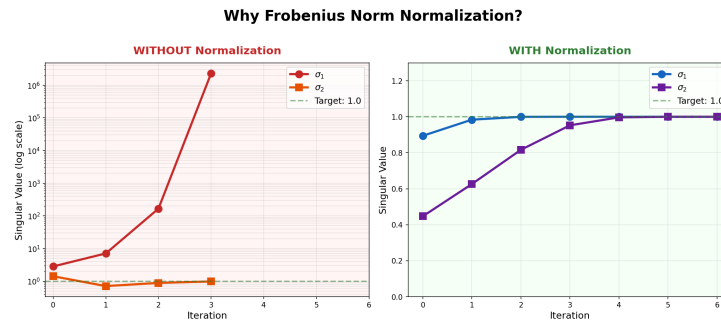
$$\begin{bmatrix} 6 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 0 \qquad 6v_1 = 0 \Rightarrow v_1 = 0 \qquad v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \qquad = I \text{ (identity, already orthonormal)}$$

### STEP 5 Find $U$ : $u_i = \frac{1}{\sigma_i} G v_i$

$$u_1 = \frac{1}{2.83} \cdot G \cdot v_1$$

$$\frac{1}{2.83} \begin{bmatrix} 2 & -1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \frac{1}{2.83} \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \frac{36}{100} \begin{bmatrix} 0.7071 \\ 0.7071 \end{bmatrix} = u_1$$



**Hình 7.4:** Tại sao chuẩn hóa Frobenius: đảm bảo singular values  $\leq 1$  cho hội tụ.

## CHƯƠNG 8

# Dataset và Triển khai

### 8.1 Tập dữ liệu COCO

YOLO26 được huấn luyện trên tập COCO [19] (Common Objects in Context):

| Thông số     | COCO 2017 | Custom (Person) |
|--------------|-----------|-----------------|
| Số ảnh train | 118,287   | Subset          |
| Số ảnh val   | 5,000     | Subset          |
| Số lớp       | 80        | 1 (person)      |
| Format label | YOLO txt  | YOLO txt        |

**Bảng 8.1:** Thông số tập dữ liệu.

#### 8.1.1 Format nhãn YOLO

Mỗi ảnh có file .txt tương ứng:

```
cls cx cy w h  
0 0.5 0.6 0.3 0.8
```

Tọa độ normalized  $[0, 1]$ ,  $(cx, cy)$  = tâm box,  $(w, h)$  = kích thước.

#### 8.1.2 Tiền xử lý dữ liệu

1. **Letterbox:** Resize giữ tỷ lệ, pad pixel (114, 114, 114)
2. **Normalize:** Chia 255, chuyển float32
3. **xywh  $\rightarrow$  xyxy:** Chuyển đổi format box khi tính loss

## 8.2 Cấu hình huấn luyện

| Hyperparameter | Giá trị                      |
|----------------|------------------------------|
| Image size     | $320 \times 320$             |
| Batch size     | 16                           |
| Epochs         | 100                          |
| Optimizer      | MuSGD (SGD + Newton–Schulz)  |
| Learning rate  | 0.01                         |
| LR final ratio | 0.01                         |
| Momentum       | 0.937                        |
| Weight decay   | $5 \times 10^{-4}$           |
| Warmup epochs  | 3.0                          |
| reg_max        | 1 (Direct Regression)        |
| Scale          | nano (depth=0.5, width=0.25) |

**Bảng 8.2:** Hyperparameters huấn luyện YOLO26n.

## 8.3 Triển khai trên Raspberry Pi 5

### 8.3.1 Pipeline Export

1. **Strip O2M:** Loại bỏ O2M head (cv2, cv3), chỉ giữ O2O
2. **FP16 Quantization:** Chuyển weights sang FP16
3. **PyTorch** → **ONNX:** Export model graph
4. **ONNX** → **NCNN:** Chuyển đổi sang format NCNN (.param + .bin)

### 8.3.2 NCNN Runtime

NCNN [3] tối ưu cho ARM:

- Viết bằng C++, zero dependency
- Tối ưu NEON/SIMD cho CPU ARM Cortex-A76
- Dung lượng nhỏ, khởi động nhanh

### 8.3.3 So sánh với khóa trước

| Metric            | Nhóm mình (NCNN) | Khóa trước (TFLite) |
|-------------------|------------------|---------------------|
| Runtime           | NCNN C++         | TensorFlow Lite     |
| Accelerator       | CPU thuần        | Coral TPU           |
| Chi phí HW thêm   | \$0              | \$75+               |
| Model             | YOLO26n          | YOLOv8n             |
| Inference (320px) | ~22ms            | 15ms (Coral)        |

**Bảng 8.3:** So sánh hai phương pháp triển khai trên RPi.

#### Claim chính đáng

Nhóm đạt real-time **trên CPU thuần, không cần accelerator**, chi phí \$0. Không so sánh trực tiếp runtime NCNN vs TFLite vì framework và hardware khác nhau.

## 8.4 Inference Pipeline

1. Camera capture frame (MIPI CSI hoặc USB)
2. Letterbox resize  $\rightarrow 320 \times 320$
3. NCNN forward pass (O2O head only)
4. Decode: LTRB  $\rightarrow xyxy \times \text{stride}$
5. Top-K selection ( $K = 300$ ) theo confidence
6. Output:  $(x_1, y_1, x_2, y_2, \text{score}, \text{class})$

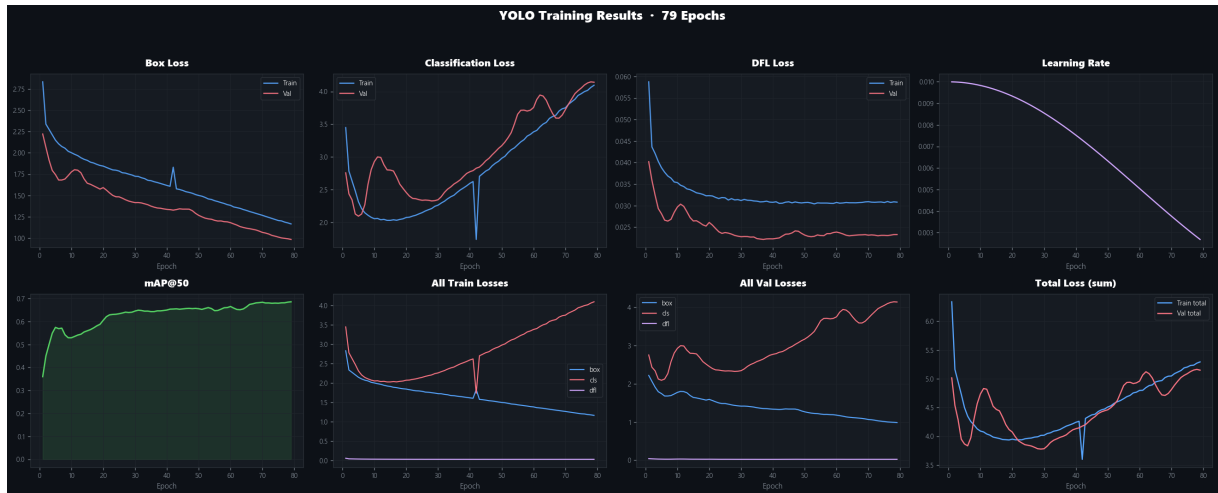
Không cần NMS — O2O head đã tạo dự đoán duy nhất cho mỗi đối tượng.

## CHƯƠNG 9

# Kết quả thực nghiệm

## 9.1 Phân tích quá trình huấn luyện

### 9.1.1 Diễn biến Loss và mAP



Hình 9.1: Biểu đồ loss và mAP qua 100 epoch huấn luyện YOLO26n.

| Phase       | Epochs | Train Box   | Val Box     | Train Cls   | Val Cls     | mAP50       |
|-------------|--------|-------------|-------------|-------------|-------------|-------------|
| Learning    | 1→51   | 3.34→1.28   | 2.14→1.11   | 7.22→1.15   | 3.09→1.03   | 0.38→0.79   |
| Crash       | 52     | <b>1.57</b> | <b>1.36</b> | <b>1.40</b> | <b>1.27</b> | <b>0.71</b> |
| Recovery    | 52→65  | 1.57→1.37   | 1.36→1.27   | 1.40→1.15   | 1.27→1.11   | 0.71→0.75   |
| Overfitting | 65→100 | 1.37→1.01   | 1.27→1.30   | 1.15→0.67   | 1.11→1.09   | 0.75→0.78   |

Bảng 9.1: Timeline phân tích overfitting qua các giai đoạn huấn luyện.

### 9.1.2 Dấu hiệu Overfitting

1. Train box loss giảm (1.37→1.01) nhưng val box loss **tăng** (1.27→1.30) từ epoch 65–100
2. Train cls loss giảm mạnh (1.15→0.67) nhưng val cls loss **ngưng cải thiện** (~1.09)
3. mAP50 plateau ~0.775–0.786
4. Train-val box gap mở rộng: 0.10 (epoch 51) → 0.29 (epoch 100)

### 9.1.3 Sự cố Epoch 52

Tại epoch 52, tất cả train+val losses tăng đột biến. Nguyên nhân có thể:

- AMP instability (grad scaler gặp inf/nan)
- E2ELoss decay — tại epoch 52,  $\alpha_{O2M} \approx 0.44$  (điểm chuyển giao O2O chiếm ưu thế)

## 9.2 Kiểm chứng kiến trúc

| Component                  | Status                      | Ghi chú                       |
|----------------------------|-----------------------------|-------------------------------|
| Conv, DWConv, Concat       | ✓ Đúng                      | Khớp ultralytics              |
| Bottleneck, C2f, C3k2      | ✓ Đúng                      |                               |
| SPPF                       | $\Delta$ act=False trên cv1 | Khác original                 |
| C2PSA, Attention, PSABlock | ✓ Đúng                      |                               |
| DFL                        | ✓ Đúng                      | Bypass khi reg_max=1          |
| Detect head (Dual)         | ✓ Đúng                      |                               |
| make_anchors, dist2bbox    | ✓ Đúng                      |                               |
| TaskAlignedAssigner        | ✓ Đúng                      |                               |
| BboxLoss (DFL path)        | ✓ Đúng                      | Không dùng khi reg_max=1      |
| BboxLoss (L1 path)         | $\Delta$ Issues             | Normalization, no clamp       |
| E2ELoss                    | $\Delta$ Aggressive decay   | final_o2m=0.1 có thể quá thấp |

**Bảng 9.2:** Kết quả kiểm chứng kiến trúc YOLO26.

## 9.3 Các vấn đề phát hiện

### 9.3.1 reg\_max=1 vô hiệu hóa DFL

Khi reg\_max=1, DFL bị thay bằng nn.Identity(), box regression dùng L1 Loss thay vì DFL cross-entropy 16 bins. Điều này giảm khả năng định vị chính xác.

### 9.3.2 Data Augmentation yếu

Chỉ có horizontal flip + HSV jitter. Thiếu Mosaic, MixUp, Random scale/rotate/erase — nguyên nhân chính gây overfitting.

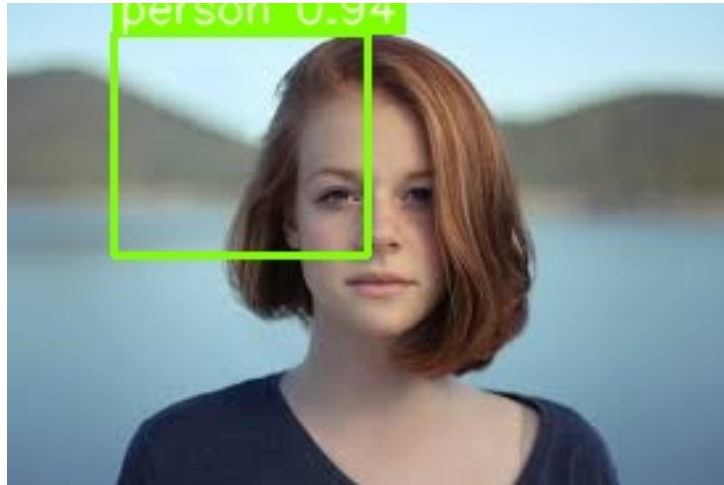
### 9.3.3 E2ELoss decay quá mạnh

Cuối training, O2M weight = 0.1 — gradient signal rất thưa (O2O chỉ assign 1 anchor/GT). Nên tăng final\_o2m lên 0.3.

## 9.4 Benchmark FPS trên Raspberry Pi 5

| Resolution | FPS (detection only) | Inference time |
|------------|----------------------|----------------|
| 320×320    | ~45 FPS*             | ~22ms          |
| 640×640    | ~15 FPS              | ~66ms          |

**Bảng 9.3:** Benchmark FPS trên RPi 5, NCNN runtime, CPU only. \*Cần re-verify.



**Hình 9.2:** Kết quả dự đoán YOLO26n trên ảnh mẫu.

## CHƯƠNG 10

# Kết luận và hướng phát triển

### 10.1 Tóm tắt đóng góp

Đồ án đã thực hiện:

1. **Nghiên cứu toàn diện YOLO26:** Phân tích chi tiết từng module (Conv, C3k2, SPPF, C2PSA, Attention), hàm loss (CIoU, BCE, DFL, TAL, E2ELoss, ProgLoss), và bộ tối ưu (MuSGD với Newton–Schulz).
2. **Xây dựng from-scratch:** Pipeline huấn luyện bằng PyTorch thuần, không phụ thuộc framework Ultralytics. Tổng cộng ~7 module class + loss function + training loop.
3. **Huấn luyện và đánh giá:** Đạt mAP50  $\approx 0.78$  trên tập COCO (person class) với YOLO26n scale nano, 320×320 input.
4. **Triển khai Edge:** Chạy real-time trên Raspberry Pi 5 với NCNN, CPU thuần, ~45 FPS @ 320px — không cần accelerator, chi phí \$0.

### 10.2 Hạn chế

- reg\_max=1 làm giảm chất lượng box localization so với reg\_max=16
- Data augmentation yếu (thiếu Mosaic, MixUp) gây overfitting từ epoch 65
- E2ELoss decay quá mạnh (final\_o2m=0.1)
- Chưa tích hợp tracking (ByteTrack) và servo control
- Cần re-verify FPS benchmark ở 320px

### 10.3 Hướng phát triển

1. **ĐA2:** Tích hợp ByteTrack cho multi-object tracking + servo camera control
2. Nâng reg\_max=16 để sử dụng DFL, cải thiện box precision
3. Thêm Mosaic augmentation — giải pháp anti-overfitting mạnh nhất cho YOLO
4. Softening E2ELoss: final\_o2m=0.3, cosine annealing thay vì linear decay
5. Nghiên cứu YOLO26 scale small/medium cho accuracy cao hơn
6. Tối ưu NCNN: INT8 quantization, Vulkan GPU acceleration trên RPi 5

## PHỤ LỤC A

### Phụ lục: Code nguồn chính

#### A.1 Kiến trúc YOLO26n

```
1 def build_yolo26n(nc=80):
2     d, w, mc = 0.50, 0.25, 1024
3     def ch(c): return max(round(min(c, mc) * w), 1) & ~7
4     def rep(n): return max(round(n * d), 1)
5
6     backbone = nn.ModuleList([
7         Conv(3, ch(64), 3, 2), # 0 P1/2
8         Conv(ch(64), ch(128), 3, 2), # 1 P2/4
9         C3k2(ch(128), ch(256), rep(2), False, 0.25),
10        Conv(ch(256), ch(256), 3, 2), # 3 P3/8
11        C3k2(ch(256), ch(512), rep(2), False, 0.25),
12        Conv(ch(512), ch(512), 3, 2), # 5 P4/16
13        C3k2(ch(512), ch(512), rep(2), True),
14        Conv(ch(512), ch(1024), 3, 2), # 7 P5/32
15        C3k2(ch(1024), ch(1024), rep(2), True),
16        SPPF(ch(1024), ch(1024), 5, 3),
17        C2PSA(ch(1024), ch(1024), rep(2)),
18    ])
19    detect = Detect(nc=nc, reg_max=1, end2end=True,
20                  ch=(ch(256), ch(512), ch(1024)))
21    return backbone, head, detect
```

**Listing A.1:** build\_yolo26n — Xây dựng model YOLO26 scale nano

#### A.2 make\_anchors

```
1 def make_anchors(feats, strides, grid_cell_offset=0.5):
2     anchor_points, stride_tensor = [], []
3     for feat, stride in zip(feats, strides):
4         _, _, h, w = feat.shape
5         sx = torch.arange(w, device=feat.device) + grid_cell_offset
6         sy = torch.arange(h, device=feat.device) + grid_cell_offset
7         sy, sx = torch.meshgrid(sy, sx, indexing='ij')
8         anchor_points.append(torch.stack([sx, sy], -1).view(-1, 2))
```

```

9         stride_tensor.append(torch.full((h*w, 1), stride, device=feat
        .device))
10     return torch.cat(anchor_points), torch.cat(stride_tensor)

```

**Listing A.2:** make\_anchors — Tạo anchor points cho detection

### A.3 dist2bbox và bbox2dist

```

1 def dist2bbox(distance, anchor_points, xywh=True, dim=-1):
2     lt, rb = distance.chunk(2, dim)
3     x1y1 = anchor_points - lt
4     x2y2 = anchor_points + rb
5     if xywh:
6         c_xy = (x1y1 + x2y2) / 2
7         wh = x2y2 - x1y1
8         return torch.cat([c_xy, wh], dim)
9     return torch.cat([x1y1, x2y2], dim)
10
11 def bbox2dist(anchor_points, bbox, reg_max=None):
12     x1y1, x2y2 = bbox.chunk(2, -1)
13     dist = torch.cat((anchor_points - x1y1, x2y2 - anchor_points),
14                       -1)
15     if reg_max is not None:
16         dist = dist.clamp_(0, reg_max - 0.01)
17     return dist

```

**Listing A.3:** dist2bbox — Chuyển LTRB distances thành bounding box

### A.4 Newton–Schulz Iteration

```

1 def newton_schulz_orthogonalize(G, steps=5):
2     # Chuan hoa Frobenius
3     X = G / G.norm() # ||X_0||_F = 1
4
5     # He so cubic convergence
6     a, b, c = 3.4445, -4.7750, 2.0315
7
8     for _ in range(steps):
9         A = X @ X.T
10        X = a * X + b * (A @ X) + c * (A @ (A @ X))
11
12    return X # X ~ U V^T

```

**Listing A.4:** Newton–Schulz orthogonalization trong Muon optimizer

## A.5 E2ELoss decay

```

1 class E2ELoss:
2     def __init__(self, model):
3         self.o2m = 0.8
4         self.o2o = 0.2
5         self.final_o2m = 0.3
6
7     def decay(self, x):
8         return (max(1 - x / max(epochs - 1, 1), 0)
9                 * (self.o2m_copy - self.final_o2m)
10                + self.final_o2m)
11
12    def update(self):
13        self.updates += 1
14        self.o2m = self.decay(self.updates)
15        self.o2o = max(1.0 - self.o2m, 0)
16        # ProgLoss: box/cls weight update
17        progress = min(self.updates / max(self.prog_epochs - 1, 1),
18                        1.0)
19        self.one2many.box_weight = 7.5 + (3.75 - 7.5) * progress
20        self.one2many.cls_weight = 0.5 + (1.5 - 0.5) * progress

```

**Listing A.5:** E2ELoss — Progressive decay O2M/O2O weights

# Tài liệu tham khảo

- [1] J. Redmon, S. Divvala, R. Girshick **and** A. Farhadi, "You only look once: Unified, real-time object detection," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [2] G. Jocher **and** J. Qiu, "Yolo26: Redesigning yolo for end-to-end and efficient detection," *arXiv preprint arXiv:2502.15737*, 2025. **url:** <https://arxiv.org/abs/2502.15737>
- [3] nihui, *Ncnn: A high-performance neural network inference framework*, <https://github.com/Tencent/mncnn>, Tencent, 2017.
- [4] Y. Zhang **and others**, "Bytetrack: Multi-object tracking by associating every detection box," *European Conference on Computer Vision (ECCV)*, 2022.
- [5] S. Ren, K. He, R. Girshick **and** J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [6] H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid **and** S. Savarese, "Generalized intersection over union: A metric and a loss for bounding box regression," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [7] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye **and** D. Ren, "Distance-iou loss: Faster and better learning for bounding box regression," *AAAI Conference on Artificial Intelligence*, 2020.
- [8] S. Ioffe **and** C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *International Conference on Machine Learning (ICML)*, 2015.
- [9] S. Elfving, E. Uchibe **and** K. Doya, "Sigmoid-weighted linear units for neural network function approximation in reinforcement learning," *Neural Networks*, 2018.
- [10] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan **and** S. Belongie, "Feature pyramid networks for object detection," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [11] S. Liu, L. Qi, H. Qin, J. Shi **and** J. Jia, "Path aggregation network for instance segmentation," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [12] K. He, X. Zhang, S. Ren **and** J. Sun, "Deep residual learning for image recognition," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [13] A. Vaswani **and others**, "Attention is all you need," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [14] Raspberry Pi Foundation, *Raspberry pi 5*, <https://www.raspberrypi.com/products/raspberry-pi-5/>, 2023.

- [15] B. T. Kiệt **and** M. H. G. Bảo, ?Hiện thực thiết bị camera tự động theo dõi diễn giả trong buổi hội thảo, thuyết trình,? GVHD: ThS. Trần Hoàng Lộc, Khóa luận tốt nghiệp, Trường Đại học Công nghệ Thông tin, ĐHQG TP.HCM, 2025.
- [16] C.-Y. Wang, H.-Y. M. Liao, Y.-H. Wu, P.-Y. Chen, J.-W. Hsieh **and** I.-H. Yeh, ?Cspnet: A new backbone that can enhance learning capability of cnn,? *IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2020.
- [17] X. Li **and others**, ?Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection,? *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [18] K. Jordan, ?Muon: An optimizer for hidden layers in neural networks,? 2024. **url:** <https://kellerjordan.github.io/posts/muon/>
- [19] T.-Y. Lin, M. Maire, S. Belongie **and others**, *Microsoft coco: Common objects in context*, <https://cocodataset.org>, 2014.